

KOTLIN & ANDROID DECLARATIONS (ANNOTATED)

archive/ui/ActiveSessionViewModelFactory.kt

```
L11: class ActiveSessionViewModelFactory(    Class    app component; see source for details.
```

archive/ui/SessionsViewModelFactory.kt

```
L10: class SessionsViewModelFactory(    Class    app component; see source for details.
```

archive/ui/ExtractionConfigScreen.kt

```
L20: fun ExtractionConfigScreen(    Function    performs a feature-specific operation.
```

archive/ui/BaseAiViewModel.kt

```
L13: sealed class AiState {    Sealed hierarchy    closed set of subclasses.
```

```
L14: object Idle : AiState()    Singleton object    shared utilities/DI.
```

```
L15: object Loading : AiState()    Singleton object    shared utilities/DI.
```

```
L16: data class Success(val result: String) : AiState()    Data model    holds structured fields.
```

```
L17: data class Error(val message: String) : AiState()    Data model    holds structured fields.
```

```
L37: protected fun executeAiTask(    Function    performs a feature-specific operation.
```

```
L62: fun resetAiState() {    Function    performs a feature-specific operation.
```

```
L66: fun cancelCurrentAiTask() {    Function    performs a feature-specific operation.
```

core-auth/src/main/java/com/algorithmx/q_base/data/di/AuthModule.kt

```
L13: object AuthModule {    Singleton object    shared utilities/DI.
```

```
L17: fun provideAppwriteAccount(client: Client): Account = Account(client)    Function
performs a feature-specific operation.
```

core-auth/src/main/java/com/algorithmx/q_base/data/auth/UserProfile.kt

```
L7: data class UserProfile(    Data model    holds structured fields.
```

core-auth/src/main/java/com/algorithmx/q_base/data/auth/ProfileRepository.kt

```
L14: class ProfileRepository @Inject constructor(    Repository    coordinates data sources for
a feature.
```

```
L21: private fun mapToUserProfile(map: Map<String, Any>): UserProfile {    Function    performs
a feature-specific operation.
```

```
L35: private fun userProfileToMap(profile: UserProfile): Map<String, Any> {    Function
performs a feature-specific operation.
```

```
L48: suspend fun createOrUpdateProfile(    Creates resources/documents.
```

```
L94: suspend fun updateProfile(profile: UserProfile): Result<Unit> {    Function    performs a
feature-specific operation.
```

```
L107: private suspend fun saveProfile(profile: UserProfile, email: String) {    Persists or
updates data.
```

```

L129: suspend fun findUserByFriendCode(friendCode: String): Result<UserProfile?> {    Function
    performs a feature-specific operation.

L148: suspend fun syncUserProfile(userId: String) {    Function    performs a feature-specific
operation.

L219: private suspend fun generateUniqueFriendCode(): String {    Generates content (often
AI).

L238: private suspend fun isFriendCodeUnique(friendCode: String): Boolean {    Function
performs a feature-specific operation.

L251: private fun generateFriendCode(): String {    Generates content (often AI).

L259: suspend fun setupSecureBackup(userId: String, passphrase: String): Result<Unit> {
Persists or updates data.

L277: suspend fun checkHasSecureBackup(userId: String): Boolean {    Function    performs a
feature-specific operation.

L287: suspend fun restoreSecureBackup(userId: String, passphrase: String): Result<Unit> {
Function    performs a feature-specific operation.

```

core-auth/src/main/java/com/algorithmx/q_base/data/auth/ProfileCache.kt

```

L3: interface ProfileCache {    Interface    contract for implementations.

L4: suspend fun upsert(profile: UserProfile)    Function    performs a feature-specific
operation.

```

core-auth/src/main/java/com/algorithmx/q_base/data/auth/AuthRepository.kt

```

L23: data class AppwriteUser(    Data model    holds structured fields.

L31: class AuthRepository @Inject constructor(    Repository    coordinates data sources for a
feature.

L52: private fun saveUserToPrefs(user: AppwriteUser) {    Persists or updates data.

```

```

L62: private fun getUserFromPrefs(): AppwriteUser? {    Getter/fetcher  retrieves data.

L71: private fun clearUserFromPrefs() {    Function  performs a feature-specific operation.

L81: fun checkCurrentSession() {    Function  performs a feature-specific operation.

L103: private fun mapAppwriteUser(user: User<*>, photoUrl: String? = null): AppwriteUser {
Function  performs a feature-specific operation.

L116: private suspend fun fetchGooglePhotoUrl(): String? {    Function  performs a
feature-specific operation.

L153: suspend fun signInWithEmail(email: String, pass: String): Result<AppwriteUser> {
Function  performs a feature-specific operation.

L168: suspend fun signUpWithEmail(email: String, pass: String, username: String, photoUrl:
String? = null): Result<AppwriteUser> {    Function  performs a feature-specific operation.

L184: suspend fun signInWithGoogle(activity: ComponentActivity): Result<AppwriteUser> {
Function  performs a feature-specific operation.

L201: fun signOut() {    Function  performs a feature-specific operation.

```

core-auth/src/main/java/com/algorithmx/q_base/data/backend/FirebaseAuthImpl.kt

```

L21: class FirebaseAuthImpl @Inject constructor(    Class  app component; see source for
details.

```

core-auth/src/main/java/com/algorithmx/q_base/data/backend/CoreDatabase.kt

```

L5: enum class CoreQueryOperator {    Enumeration  named constants.

L13: data class CoreQuery(    Data model  holds structured fields.

L19: interface CoreDatabase {    Interface  contract for implementations.

L20: suspend fun createDocument(collectionId: String, documentId: String, data: Map<String,

```

Any>): Result<Unit> Creates resources/documents.

L21: suspend fun getDocument(collectionId: String, documentId: String): Result<Map<String, Any>?> Getter/fetcher retrieves data.

L22: suspend fun updateDocument(collectionId: String, documentId: String, data: Map<String, Any>): Result<Unit> Function performs a feature-specific operation.

L23: suspend fun deleteDocument(collectionId: String, documentId: String): Result<Unit> Function performs a feature-specific operation.

L24: suspend fun listDocuments(collectionId: String): Result<List<Map<String, Any>>> Function performs a feature-specific operation.

L25: suspend fun queryDocuments(collectionId: String, queries: List<CoreQuery>): Result<List<Map<String, Any>>> Function performs a feature-specific operation.

L26: fun observeDocument(collectionId: String, documentId: String): Flow<Map<String, Any>?> Returns observable/Flow for reactive updates.

L27: fun observeCollection(collectionId: String): Flow<List<Map<String, Any>>> Returns observable/Flow for reactive updates.

core-auth/src/main/java/com/algorithmx/q_base/data/backend/Appwrite DatabaseImpl.kt

L19: class AppwriteDatabaseImpl @Inject constructor(Class app component; see source for details.

L30: private suspend fun invokeSuspendReflective(Function performs a feature-specific operation.

L56: private fun mapRow(row: Any): Map<String, Any> { Function performs a feature-specific operation.

L132: private fun mapRowList(rowList: Any): List<Map<String, Any>> { Function performs a feature-specific operation.

L143: private fun mapDocument(doc: io.appwrite.models.Document<*>): Map<String, Any> { Function performs a feature-specific operation.

L393: fun wrapValue(v: Any): List<Any> = if (v is List<*>) v.filterNotNull() else listOf(v) Function performs a feature-specific operation.

core-auth/src/main/java/com/algorithmx/q_base/data/backend/FirebaseDatabaseImpl.kt

L13: class FirebaseDatabaseImpl @Inject constructor(Class app component; see source for details.

core-auth/src/main/java/com/algorithmx/q_base/data/backend/BackendModule.kt

L16: annotation class FirebaseBackend Class app component; see source for details.

L20: annotation class AppwriteBackend Class app component; see source for details.

L24: object BackendModule { Singleton object shared utilities/DI.

L28: fun provideFirebaseAuth(): FirebaseAuth { Function performs a feature-specific operation.

L35: fun provideFirebaseAuthImpl(Function performs a feature-specific operation.

L45: fun provideFirebaseDatabaseImpl(Function performs a feature-specific operation.

L54: fun provideAppwriteAuthImpl(Function performs a feature-specific operation.

L64: fun provideAppwriteDatabaseImpl(Function performs a feature-specific operation.

L74: fun provideActiveAuth(Function performs a feature-specific operation.

L82: fun provideActiveDatabase(Function performs a feature-specific operation.

core-auth/src/main/java/com/algorithmx/q_base/data/backend/CoreAuth.kt

L6: data class CoreUser(Data model holds structured fields.

L13: interface CoreAuth { Interface contract for implementations.

L17: suspend fun signInWithEmail(email: String, pass: String): Result<CoreUser> Function performs a feature-specific operation.

L18: suspend fun signUpWithEmail(email: String, pass: String, username: String): Result<CoreUser> Function performs a feature-specific operation.

L19: suspend fun signInWithGoogle(activity: ComponentActivity): Result<CoreUser> Function performs a feature-specific operation.

L20: suspend fun signOut(): Result<Unit> Function performs a feature-specific operation.

L21: suspend fun checkCurrentSession(): Result<CoreUser?> Function performs a feature-specific operation.

core-auth/src/main/java/com/algorithmx/q_base/data/backend/Appwrite AuthImpl.kt

L18: class AppwriteAuthImpl @Inject constructor(Class app component; see source for details.

L37: private fun mapUser(user: User<*>): CoreUser { Function performs a feature-specific operation.

core-auth/src/main/java/com/algorithmx/q_base/ui/components/AuthComponents.kt

L45: fun AuthFormSection(Function performs a feature-specific operation.

L193: fun GoogleSignInButton(Function performs a feature-specific operation.

L284: fun GoogleIconGraphic() { Function performs a feature-specific operation.

core-chat/src/main/java/com/algorithmx/q_base/data/chat/ChatEntity.kt

L9: data class ChatEntity(Data model holds structured fields.

L22: fun ChatEntity.isAdmin(userId: String): Boolean { Function performs a feature-specific operation.

core-chat/src/main/java/com/algorithmx/q_base/data/chat/MessageEntity.kt

L20: data class MessageEntity(Data model holds structured fields.

core-chat/src/main/java/com/algorithmx/q_base/data/chat/ChatDao.kt

L6: data class ChatSummary(Data model holds structured fields.

L19: interface ChatDao { Interface contract for implementations.

L21: suspend fun insertChat(chat: ChatEntity) Function performs a feature-specific operation.

L24: suspend fun getChatById(chatId: String): ChatEntity? Getter/fetcher retrieves data.

L27: fun getChatByIdFlow(chatId: String): Flow<ChatEntity?> Getter/fetcher retrieves data.

L30: suspend fun getP2PChat(ids1: String, ids2: String): ChatEntity? Getter/fetcher retrieves data.

L33: fun getAllChats(): Flow<List<ChatEntity>> Getter/fetcher retrieves data.

L46: fun getChatSummaries(): Flow<List<ChatSummary>> Getter/fetcher retrieves data.

L49: suspend fun updateBlockedStatus(chatId: String, isBlocked: Boolean) Function performs a feature-specific operation.

L52: suspend fun updateReportedStatus(chatId: String, isReported: Boolean) Function performs a feature-specific operation.

L55: suspend fun updateMutedStatus(chatId: String, isMuted: Boolean) Function performs a feature-specific operation.

L58: suspend fun updateParticipants(chatId: String, participantIds: String) Function performs a feature-specific operation.

```

L61: suspend fun deleteChat(chat: ChatEntity)    Function performs a feature-specific
operation.

L64: suspend fun deleteChatById(chatId: String)  Function performs a feature-specific
operation.

L67: suspend fun incrementUnreadCount(chatId: String)  Function performs a feature-specific
operation.

L70: suspend fun clearUnreadCount(chatId: String)  Function performs a feature-specific
operation.

L73: fun getTotalUnreadCount(): Flow<Int?>    Getter/fetcher retrieves data.

L76: suspend fun deleteAllChats()    Function performs a feature-specific operation.

```

core-chat/src/main/java/com/algorithmx/q_base/data/chat/ChatDatabase.kt

```

L25: fun getDatabase(context: Context): ChatDatabase {    Getter/fetcher retrieves data.

```

core-chat/src/main/java/com/algorithmx/q_base/data/chat/ChatTypeConverters.kt

```

L8: object ChatTypeConverters {    Singleton object shared utilities/DI.

L13: fun fromAdminIds(adminIds: List<String>?): String {    Function performs a
feature-specific operation.

L19: fun toAdminIds(value: String?): List<String> {    Function performs a feature-specific
operation.

```

core-chat/src/main/java/com/algorithmx/q_base/data/chat/ChatRemoteRepository.kt

L14: class ChatRemoteRepository @Inject constructor(Repository coordinates data sources for a feature.

L18: private suspend fun getCurrentUserId(): String? { Getter/fetcher retrieves data.

L26: suspend fun createChatOnRemote(chat: ChatEntity) { Creates resources/documents.

L82: suspend fun addParticipantToRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L104: suspend fun removeParticipantFromRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L126: suspend fun promoteParticipantToAdminOnRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L148: suspend fun demoteAdminOnRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L170: suspend fun clearChatMessagesOnRemote(chatId: String) { Function performs a feature-specific operation.

L204: suspend fun deleteChatOnRemote(chatId: String) { Function performs a feature-specific operation.

L234: suspend fun reportGroup(group: ChatEntity, reason: String) { Function performs a feature-specific operation.

L254: suspend fun reportMessage(message: MessageEntity, reason: String) { Function performs a feature-specific operation.

core-chat/src/main/java/com/algorithmx/q_base/data/chat/MessageDao.kt

L7: interface MessageDao { Interface contract for implementations.

L9: suspend fun insertMessage(message: MessageEntity) Function performs a feature-specific operation.

L12: fun getMessagesForChat(chatId: String): Flow<List<MessageEntity>> Getter/fetcher retrieves data.

L15: suspend fun getMessageById(messageId: String): MessageEntity? Getter/fetcher retrieves data.

```

L18: fun getAllMessages(): Flow<List<MessageEntity>>    Getter/fetcher  retrieves data.

L21: suspend fun deleteMessage(message: MessageEntity)    Function  performs a
feature-specific operation.

L24: suspend fun deleteMessagesByChatId(chatId: String)    Function  performs a
feature-specific operation.

L27: suspend fun deleteAllMessages()    Function  performs a feature-specific operation.

L30: suspend fun getPendingMessages(): List<MessageEntity>    Getter/fetcher  retrieves data.

L33: suspend fun updateMessageStatus(messageId: String, status: String)    Function  performs
a feature-specific operation.

```

core-crypto/src/main/java/com/algorithmx/q_base/core_crypto/CryptoManager.kt

```

L21: class CryptoManager @Inject constructor(    Class  app component; see source for details.

L37: fun initializeAndGetPublicKey(): String {    Function  performs a feature-specific
operation.

L57: fun encryptMessage(plaintext: String, receiverPublicKeyBase64: String): String {
Function  performs a feature-specific operation.

L69: fun encryptSessionKey(sessionKey: ByteArray, receiverPublicKeyBase64: String): String {
Function  performs a feature-specific operation.

L81: fun decryptMessage(ciphertextBase64: String): Result<String> {    Function  performs a
feature-specific operation.

L91: fun decryptSessionKey(wrappedKeyBase64: String): Result<ByteArray> {    Function
performs a feature-specific operation.

L100: private fun decryptRaw(ciphertextBase64: String): ByteArray {    Function  performs a
feature-specific operation.

L122: fun clearKeys() {    Function  performs a feature-specific operation.

L130: fun getPublicKeyFingerprint(): String {    Getter/fetcher  retrieves data.

L142: fun encryptFileContent(plaintext: ByteArray): Pair<ByteArray, String> {    Function
performs a feature-specific operation.

```

```

L158: fun encryptWithSessionKey(plaintext: String): Pair<String, String> {    Function
performs a feature-specific operation.

L174: fun decryptWithSessionKey(ciphertextBase64: String, keyHandleBase64: String):
Result<String> {    Function    performs a feature-specific operation.

L190: fun decryptFileContent(ciphertext: ByteArray, keyBase64: String): ByteArray {
Function    performs a feature-specific operation.

L206: fun exportEncryptedKeyset(passphrase: String): String {    Function    performs a
feature-specific operation.

L219: fun importEncryptedKeyset(encryptedBackupBase64: String, passphrase: String):
Result<Unit> {    Function    performs a feature-specific operation.

L242: private fun deriveKey(passphrase: String, salt: ByteArray): ByteArray {    Function
performs a feature-specific operation.

L248: private fun encryptBackup(data: ByteArray, passphrase: String): String {    Function
performs a feature-specific operation.

L271: private fun decryptBackup(encryptedBase64: String, passphrase: String): ByteArray {
Function    performs a feature-specific operation.

L290: private fun getLocalAead(): Aead {    Getter/fetcher    retrieves data.

L305: fun encryptLocalString(plaintext: String): String {    Function    performs a
feature-specific operation.

L312: fun decryptLocalString(ciphertextBase64: String): Result<String> {    Function    performs
a feature-specific operation.

L325: private fun getGlobalDecryptionKey(): javax.crypto.spec.SecretKeySpec {
Getter/fetcher    retrieves data.

L340: fun encryptGlobalKey(plainText: String): String {    Function    performs a
feature-specific operation.

L355: fun decryptGlobalKey(combinedBase64: String): Result<String> {    Function    performs a
feature-specific operation.

```

**app/src/androidTest/java/com/algorithmx/q_base/ExampleInstrumented
Test.kt**

```
L17: class ExampleInstrumentedTest {    Class    app component; see source for details.

L19: fun useAppContext() {    Function    performs a feature-specific operation.
```

app/src/test/java/com/algorithmx/q_base/ExampleUnitTest.kt

```
L12: class ExampleUnitTest {    Class    app component; see source for details.

L14: fun addition_isCorrect() {    Function    performs a feature-specific operation.
```

app/src/test/java/com/algorithmx/q_base/ui/settings/ProfileViewModelTest.kt

```
L25: class ProfileViewModelTest {    Class    app component; see source for details.

L40: fun setup() {    Persists or updates data.

L66: fun tearDown() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/QbaseApplication.kt

```
L9: class QbaseApplication : Application() {    Class    app component; see source for details.
```

app/src/main/java/com/algorithmx/q_base/MainActivity.kt

```
L47: class MainActivity : ComponentActivity() {    UI component    activity/fragment for
screens.

L60: private fun checkNotificationPermission() {    Function    performs a feature-specific
operation.
```

```

L275: fun LoadingScreenCircularArchive() {    Function    performs a feature-specific operation.

L299: fun LoadingScreen() {    Function    performs a feature-specific operation.

L339: fun MainScreen(    Function    performs a feature-specific operation.

L453: fun AppNavDisplay(navigationState: NavigationState, navigator: Navigator) {    Function
performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/AppDatabase.kt

```

L50: fun getDatabase(context: Context): AppDatabase {    Getter/fetcher    retrieves data.

```

app/src/main/java/com/algorithmx/q_base/data/DatabaseSeeder.kt

```

L22: class DatabaseSeeder @Inject constructor(    Class    app component; see source for
details.

L32: private fun getColumnString(cursor: android.database.Cursor, index: Int): String? {
Getter/fetcher    retrieves data.

L36: suspend fun seedDatabaseIfNeeded() = withContext(Dispatchers.IO) {    Function    performs
a feature-specific operation.

L229: private suspend fun seedSampleChats() {    Function    performs a feature-specific
operation.

L336: private suspend fun seedAncientSriLanka() {    Function    performs a feature-specific
operation.

```

app/src/main/java/com/algorithmx/q_base/data/di/NetworkModule.kt

```

L23: object NetworkModule {    Singleton object    shared utilities/DI.

L27: fun provideOkHttpClient(): OkHttpClient {    Function    performs a feature-specific
operation.

```

L38: fun provideFirestore(): FirebaseFirestore = Firebase.firestore Function performs a feature-specific operation.

L42: fun provideAppwriteClient(@ApplicationContext context: Context): Client { Function performs a feature-specific operation.

L99: fun shieldJsonObject(obj: org.json.JSONObject) { Function performs a feature-specific operation.

L149: fun provideAppwriteStorage(client: Client): Storage { Function performs a feature-specific operation.

L155: fun provideAppwriteDatabases(client: Client): Databases { Function performs a feature-specific operation.

L161: fun provideAppwriteTables(client: Client): Any? { Function performs a feature-specific operation.

L178: fun provideNetworkMonitor(@ApplicationContext context: Context): NetworkMonitor { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/di/DatabaseModule.kt

L33: object DatabaseModule { Singleton object shared utilities/DI.

L37: fun provideDatabase(@ApplicationContext context: Context): AppDatabase { Function performs a feature-specific operation.

L42: fun provideQuestionDao(database: AppDatabase): QuestionDao = database.questionDao()
Function performs a feature-specific operation.

L45: fun provideCollectionDao(database: AppDatabase): CollectionDao =
database.collectionDao() Function performs a feature-specific operation.

L48: fun provideSessionDao(database: AppDatabase): SessionDao = database.sessionDao()
Function performs a feature-specific operation.

L51: fun provideProblemReportDao(database: AppDatabase): ProblemReportDao =
database.problemReportDao() Function performs a feature-specific operation.

L54: fun provideUserDao(database: AppDatabase): UserDao = database.userDao() Function performs a feature-specific operation.

L58: fun provideChatDatabase(@ApplicationContext context: Context): ChatDatabase { Function performs a feature-specific operation.

```

L63: fun provideChatDao(database: ChatDatabase): ChatDao = database.chatDao()    Function
performs a feature-specific operation.

L66: fun provideMessageDao(database: ChatDatabase): MessageDao = database.messageDao()
Function performs a feature-specific operation.

L69: fun provideAiResponseDao(database: AppDatabase): AiResponseDao =
database.aiResponseDao()    Function performs a feature-specific operation.

L72: fun provideBrainUsageDao(database: AppDatabase): BrainUsageDao =
database.brainUsageDao()    Function performs a feature-specific operation.

L75: fun provideCollectionVersionLedgerDao(database: AppDatabase): CollectionVersionLedgerDao
= database.collectionVersionLedgerDao()    Function performs a feature-specific operation.

L78: fun provideActionQueueDao(database: AppDatabase):
com.algorithmx.q_base.data.sync.ActionQueueDao = database.actionQueueDao()    Function
performs a feature-specific operation.

L82: fun provideExploreRepository(    Function performs a feature-specific operation.

L94: fun provideSessionRepository(    Function performs a feature-specific operation.

L105: fun provideHomeRepository(    Function performs a feature-specific operation.

L117: fun provideImportRepository(@ApplicationContext context: Context): ImportRepository {
Function performs a feature-specific operation.

L123: fun provideDataClearingRepository(    Function performs a feature-specific operation.

L141: fun provideProfileCache(userDao: UserDao): ProfileCache {    Function performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/di/BrainModule.kt

```

L14: object BrainModule {    Singleton object shared utilities/DI.

L18: fun provideAiRepository(    Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/util/TypeConverters.kt

```

L7: class TypeConverters {    Class    app component; see source for details.

L9: fun fromBrainTask(task: BrainTask): String {    Function    performs a feature-specific
operation.

L14: fun toBrainTask(name: String): BrainTask {    Function    performs a feature-specific
operation.

L23: fun fromBrainProvider(provider: BrainProvider): String {    Function    performs a
feature-specific operation.

L28: fun toBrainProvider(name: String): BrainProvider {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/util/ExportUtils.kt

```

L22: class ExportUtils @Inject constructor(    Class    app component; see source for details.

L31: suspend fun prepareZip(setWithQuestions: SetWithQuestions): File =
withContext(Dispatchers.IO) {    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/util/MockExporter.kt

```

L24: class MockExporter @Inject constructor(    Class    app component; see source for details.

L32: suspend fun exportCollection(collectionId: String): File? = withContext(Dispatchers.IO)
{    Function    performs a feature-specific operation.

L68: suspend fun exportSession(sessionId: String): File? = withContext(Dispatchers.IO) {
Function    performs a feature-specific operation.

L93: fun cleanup(file: File) {    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/util/MockDownloader.kt

L22: class MockDownloader @Inject constructor(Class app component; see source for details.

L35: suspend fun downloadAndImportMock(Function performs a feature-specific operation.

L89: suspend fun downloadAndImportSession(Function performs a feature-specific operation.

L134: private suspend fun importCollectionData(Function performs a feature-specific operation.

L185: private suspend fun importMockData(mockData: MockExport) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sessions/SessionDao.kt

L9: interface SessionDao { Interface contract for implementations.

L11: suspend fun insertSession(session: StudySession) Function performs a feature-specific operation.

L14: suspend fun insertSet(set: QuestionSet) Function performs a feature-specific operation.

L17: suspend fun updateSession(session: StudySession) Function performs a feature-specific operation.

L20: fun getAllSessions(): Flow<List<StudySession>> Getter/fetcher retrieves data.

L23: fun getOngoingSessions(): Flow<List<StudySession>> Getter/fetcher retrieves data.

L26: suspend fun getSessionById(sessionId: String): StudySession? Getter/fetcher retrieves data.

L29: fun getAllSets(): Flow<List<QuestionSet>> Getter/fetcher retrieves data.

L32: fun getRecentSets(): Flow<List<QuestionSet>> Getter/fetcher retrieves data.

L35: fun getPinnedQuestions(): Flow<List<Question>> Getter/fetcher retrieves data.

L38: suspend fun insertAttempts(attempts: List<SessionAttempt>) Function performs a feature-specific operation.

L41: fun getAttemptsForSession(sessionId: String): Flow<List<SessionAttempt>> Getter/fetcher retrieves data.

L44: suspend fun getAttemptsForSessionOnce(sessionId: String): List<SessionAttempt>

Getter/fetcher retrieves data.

L47: suspend fun updateAttempt(attempt: SessionAttempt) Function performs a feature-specific operation.

L50: suspend fun getAttempt(sessionId: String, questionId: String): SessionAttempt? Getter/fetcher retrieves data.

L53: fun getLastSessionForStudyCollection(collectionId: String): Flow<StudySession?> Getter/fetcher retrieves data.

L56: suspend fun deleteSessionsByIds(sessionIds: List<String>) Function performs a feature-specific operation.

L59: suspend fun deleteAttemptsForSessions(sessionIds: List<String>) Function performs a feature-specific operation.

L62: suspend fun deleteAllSessions() Function performs a feature-specific operation.

L65: suspend fun deleteAllAttempts() Function performs a feature-specific operation.

L68: suspend fun deleteAllSets() Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sessions/SessionExport.kt

L6: data class SessionExport(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/sessions/StudySession.kt

L10: data class StudySession(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/sessions/SessionAttempt.kt

```
L33: data class SessionAttempt(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/sessions/SessionRepository.kt

```
L19: class SessionRepository @Inject constructor(    Repository    coordinates data sources for a feature.
```

```
L25: fun getCurrentUser(userId: String): Flow<UserEntity?> =    Getter/fetcher    retrieves data.
```

```
L28: fun getAllSessions(): Flow<List<StudySession>> = sessionDao.getAllSessions()
Getter/fetcher    retrieves data.
```

```
L30: fun getAllStudyCollections(): Flow<List<StudyCollection>> =
collectionDao.getAllStudyCollections()    Getter/fetcher    retrieves data.
```

```
L32: suspend fun getStudyCollectionByIdOnce(collectionId: String): StudyCollection? =
Getter/fetcher    retrieves data.
```

```
L35: suspend fun getStudyCollectionByNameOnce(name: String): StudyCollection? =
Getter/fetcher    retrieves data.
```

```
L38: fun getAllSets(): Flow<List<QuestionSet>> = sessionDao.getAllSets()    Getter/fetcher
retrieves data.
```

```
L40: suspend fun getSessionById(sessionId: String): StudySession? =
sessionDao.getSessionById(sessionId)    Getter/fetcher    retrieves data.
```

```
L45: suspend fun createNewSession(    Creates resources/documents.
```

```
L87: suspend fun createNewSessionSmart(    Creates resources/documents.
```

```
L107: fun getQuestionsByStudyCollection(collection: String): Flow<List<Question>> =
Getter/fetcher    retrieves data.
```

```
L110: fun getAttemptsForSession(sessionId: String): Flow<List<SessionAttempt>> =
Getter/fetcher    retrieves data.
```

```
L113: suspend fun getQuestionById(questionId: String): Question? =    Getter/fetcher
retrieves data.
```

```
L116: fun getOptionsForQuestion(questionId: String): Flow<List<QuestionOption>> =
Getter/fetcher    retrieves data.
```

```

L119: suspend fun getAnswerForQuestion(questionId: String): Answer? =    Getter/fetcher
retrieves data.

L122: suspend fun updateAttemptAndRecalculate(attempt: SessionAttempt) {    Function    performs
a feature-specific operation.

L191: suspend fun updateAttempt(attempt: SessionAttempt) {    Function    performs a
feature-specific operation.

L195: suspend fun updateSession(session: StudySession) {    Function    performs a
feature-specific operation.

L199: suspend fun saveAnswer(answer: Answer) {    Persists or updates data.

L203: suspend fun deleteSessions(sessionIds: List<String>) {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/core/UserDao.kt

```

L7: interface UserDao {    Interface    contract for implementations.

L9: suspend fun insertUser(user: UserEntity)    Function    performs a feature-specific
operation.

L12: suspend fun getUserById(userId: String): UserEntity?    Getter/fetcher    retrieves data.

L15: fun getCurrentUser(userId: String): Flow<UserEntity?>    Getter/fetcher    retrieves data.

L18: fun getAllUsers(): Flow<List<UserEntity>>    Getter/fetcher    retrieves data.

L21: suspend fun deleteAllUsers()    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/core/HomeRepository.kt

```

L18: class HomeRepository @Inject constructor(    Repository    coordinates data sources for a
feature.

L25: fun getCurrentUser(userId: String): Flow<com.algorithmx.q_base.data.core.UserEntity?> =
Getter/fetcher    retrieves data.

```

```
L28: fun getOngoingSessions(): Flow<List<StudySession>> = sessionDao.getOngoingSessions()
Getter/fetcher retrieves data.
```

```
L30: fun getPinnedQuestions(): Flow<List<Question>> = sessionDao.getPinnedQuestions()
Getter/fetcher retrieves data.
```

```
L32: fun getRecentSessions(): Flow<List<StudySession>> = sessionDao.getAllSessions()
Getter/fetcher retrieves data.
```

```
L34: fun getAllStudyCollections(): Flow<List<StudyCollection>> =
collectionDao.getAllStudyCollections() Getter/fetcher retrieves data.
```

```
L36: fun getAllStudyCollectionsWithCount(): Flow<List<StudyCollectionWithCount>> =
Getter/fetcher retrieves data.
```

```
L39: fun getTotalUnreadCount(): Flow<Int> = chatDao.getTotalUnreadCount().map { it ?: 0 }
Getter/fetcher retrieves data.
```

app/src/main/java/com/algorithmx/q_base/data/core/UserEntity.kt

```
L7: data class UserEntity( Data model holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/core/ConfigRepository.kt

```
L19: class ConfigRepository @Inject constructor( Repository coordinates data sources for a
feature.
```

```
L47: suspend fun saveGeminiKey(key: String) { Persists or updates data.
```

```
L54: suspend fun saveGroqKey(key: String) { Persists or updates data.
```

```
L61: suspend fun fetchRemoteConfig() { Function performs a feature-specific operation.
```

```
L100: suspend fun getGeminiKeyDirectly(): String = geminiApiKey.first() Getter/fetcher
retrieves data.
```

```
L101: suspend fun getGroqKeyDirectly(): String = groqApiKey.first() Getter/fetcher
retrieves data.
```

L103: suspend fun backupKeysToCloud(userId: String) { Function performs a feature-specific operation.

L137: suspend fun restoreKeysFromCloud(userId: String) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/core/DataClearingRepository.kt

L17: class DataClearingRepository @Inject constructor(Repository coordinates data sources for a feature.

L28: suspend fun clearAllData(clearCollections: Boolean) = withContext(Dispatchers.IO) {
Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/ai/BrainUsageEntity.kt

L9: data class BrainUsageEntity(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/ai/AiResponseDao.kt

L7: interface AiResponseDao { Interface contract for implementations.

L9: fun getAllResponses(): Flow<List<AiResponseEntity>> Getter/fetcher retrieves data.

L12: suspend fun getResponseById(responseId: String): AiResponseEntity? Getter/fetcher
retrieves data.

L15: suspend fun insertResponse(response: AiResponseEntity) Function performs a feature-specific operation.

L18: suspend fun updateResponse(response: AiResponseEntity) Function performs a feature-specific operation.

L21: suspend fun deleteResponseById(responseId: String) Function performs a feature-specific operation.

L24: suspend fun deleteAllAiResponses() Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/ai/AiModuleIntegrator.kt

L17: class BrainConfigProviderImpl @Inject constructor(Class app component; see source for details.

L32: class AiUsageLoggerImpl @Inject constructor(AI component orchestrates LLM/AI requests.

app/src/main/java/com/algorithmx/q_base/data/ai/AiResponseEntity.kt

L6: data class AiResponseEntity(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/ai/AiRepository.kt

L15: class AiRepository @Inject constructor(Repository coordinates data sources for a feature.

L24: suspend fun generateCollection(Generates content (often AI).

L98: suspend fun saveAsSet(Persists or updates data.

L118: suspend fun saveAsCollection(Persists or updates data.

L150: private suspend fun saveQuestionsToSet(questions: List<com.algorithmx.q_base.core_ai.brain.models.AiQuestion>, setId: String, collectionName: String) { Persists or updates data.

L191: suspend fun extractQuestionsFromText(Function performs a feature-specific operation.

L265: suspend fun promoteAiResponseToDatabase(Function performs a feature-specific operation.

L294: suspend fun assistQuestionEditing(stem: String, options: String = "", context: String = ""): Result<String> { Function performs a feature-specific operation.

L308: suspend fun getAiAssistance(prompt: String): Result<String> { Getter/fetcher
retrieves data.

L312: suspend fun getAiResponseById(responseId: String): AiResponseEntity? { Getter/fetcher
retrieves data.

L316: private fun extractJsonFromResponse(response: String): String { Function performs a
feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/ai/BrainUsageDao.kt

L11: interface BrainUsageDao { Interface contract for implementations.

L13: suspend fun insertUsageRecord(record: BrainUsageEntity) Function performs a
feature-specific operation.

L16: fun getRecentUsage(limit: Int = 100): Flow<List<BrainUsageEntity>> Getter/fetcher
retrieves data.

L19: fun getTotalSuccessfulTokens(): Flow<Int?> Getter/fetcher retrieves data.

L22: fun getTotalTokensForTask(taskId: BrainTask): Flow<Int?> Getter/fetcher retrieves
data.

L25: suspend fun pruneOldRecords(olderThanMs: Long) Function performs a feature-specific
operation.

L28: suspend fun deleteAllBrainUsage() Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/ChatManagerRepository.kt

L23: class ChatManagerRepository @Inject constructor(Repository coordinates data sources
for a feature.

L37: suspend fun addParticipantToRemote(chatId: String, userId: String) { Function
performs a feature-specific operation.

```

L41: suspend fun removeParticipantFromRemote(chatId: String, userId: String) {    Function
performs a feature-specific operation.

L45: suspend fun promoteParticipantToAdminOnRemote(chatId: String, userId: String) {
Function    performs a feature-specific operation.

L49: suspend fun demoteAdminOnRemote(chatId: String, userId: String) {    Function    performs a
feature-specific operation.

L53: suspend fun createChatOnRemote(chat: ChatEntity) {    Creates resources/documents.

L57: suspend fun getChatById(chatId: String): ChatEntity? {    Getter/fetcher    retrieves data.

L61: suspend fun syncUserChatsFromRemote() {    Function    performs a feature-specific
operation.

L126: suspend fun findExistingP2PChat(uid: String, userId: String): ChatEntity? {    Function
performs a feature-specific operation.

L172: suspend fun ensureChatExistsLocally(chatId: String, senderId: String? = null) {
Function    performs a feature-specific operation.

L198: suspend fun fetchAndSyncChatMetadata(chatId: String) {    Function    performs a
feature-specific operation.

L237: suspend fun deleteChatOnRemote(chatId: String) {    Function    performs a
feature-specific operation.

L241: fun deleteChatAndMessagesGlobally(chatId: String) {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/sync/MessageSyncRepository.kt

```

L21: class MessageSyncRepository @Inject constructor(    Repository    coordinates data sources
for a feature.

L39: fun serializeWrappedKeys(map: Map<String, String>): String {    Function    performs a
feature-specific operation.

L45: fun deserializeWrappedKeys(jsonStr: String?): Map<String, String> {    Function    performs
a feature-specific operation.

L61: suspend fun acknowledgeMessageDelivery(    Function    performs a feature-specific

```

```
operation.
```

app/src/main/java/com/algorithmx/q_base/data/sync/OfflineActionEntity.kt

```
L8: data class OfflineActionEntity(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/sync/MessageSyncIncomingGlobalExtensions.kt

```
L17: fun MessageSyncRepository.observeAllIncomingMessages(notificationHelper:
NotificationHelper): Flow<Unit> {    Returns observable/Flow for reactive updates.
```

```
L205: suspend fun MessageSyncRepository.fetchAndSyncMessages(chatId: String) {    Function
performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/data/sync/CollectionSyncRequestExtensions.kt

```
L12: suspend fun CollectionSyncRepository.sendSyncRequest(targetUserId: String,
targetCollectionId: String) {    Function    performs a feature-specific operation.
```

```
L34: fun CollectionSyncRepository.observeIncomingRequests(): Flow<List<SyncRequest>> =
callbackFlow {    Returns observable/Flow for reactive updates.
```

```
L86: suspend fun CollectionSyncRepository.requestCollectionAccess(chatId: String,
collectionId: String) {    Function    performs a feature-specific operation.
```

```
L107: fun CollectionSyncRepository.observeAccessRequests(chatId: String):
Flow<List<Map<String, Any>>> {    Returns observable/Flow for reactive updates.
```

```
L155: suspend fun CollectionSyncRepository.grantCollectionAccess(chatId: String,
collectionId: String, requesterId: String) {    Function    performs a feature-specific
operation.
```

app/src/main/java/com/algorithmx/q_base/data/sync/MessageSyncOutgoingExtensions.kt

L11: suspend fun MessageSyncRepository.sendMessage(message: MessageEntity) { Function performs a feature-specific operation.

L143: suspend fun MessageSyncRepository.flushQueue() { Function performs a feature-specific operation.

L161: suspend fun MessageSyncRepository.clearChatMessagesOnRemote(chatId: String) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/SyncRequest.kt

L6: data class SyncRequest(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/sync/CollectionSyncPatchExtensions.kt

L15: suspend fun CollectionSyncRepository.applyCollectionMicroUpdate(payload: String) { Function performs a feature-specific operation.

L128: suspend fun CollectionSyncRepository.broadcastCollectionMicroUpdate(chatId: String, collectionId: String, diff: JSONObject) { Function performs a feature-specific operation.

L168: suspend fun CollectionSyncRepository.sendCollectionPatch(chatId: String, collectionId: String, op: String, data: JSONObject) { Function performs a feature-specific operation.

L185: suspend fun CollectionSyncRepository.applyCollectionPatch(jsonString: String) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/CollectionSyncRepository.kt

L23: class CollectionSyncRepository @Inject constructor(Repository coordinates data sources for a feature.

app/src/main/java/com/algorithmx/q_base/data/sync/MessageSyncIncomingExtensions.kt

L14: fun MessageSyncRepository.observeAndSyncMessages(chatId: String): Flow<MessageEntity?> { Returns observable/Flow for reactive updates.

app/src/main/java/com/algorithmx/q_base/data/sync/CollectionSyncFileExtensions.kt

L19: suspend fun CollectionSyncRepository.uploadQuestionBankZip(zipFile: File): Pair<String, String> { Function performs a feature-specific operation.

L39: suspend fun CollectionSyncRepository.deleteQuestionBankZip(fileId: String) { Function performs a feature-specific operation.

L48: suspend fun CollectionSyncRepository.shareCollectionToGroup(chatId: String, collectionMetadata: Map<String, Any>) { Function performs a feature-specific operation.

L165: suspend fun CollectionSyncRepository.acknowledgeCollectionDownload(collectionId: String) { Function performs a feature-specific operation.

L203: fun CollectionSyncRepository.observeGroupLibrary(chatId: String): Flow<List<Map<String, Any>>> { Returns observable/Flow for reactive updates.

L235: private suspend fun CollectionSyncRepository.mapGroupLibrary(list: List<Map<String, Any>>): List<Map<String, Any>> { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/SyncRepository.kt

L18: class SyncRepository @Inject constructor(Repository coordinates data sources for a feature.

L28: fun observeAndSyncMessages(chatId: String): Flow<MessageEntity?> { Returns observable/Flow for reactive updates.

L32: suspend fun sendMessage(message: MessageEntity) { Function performs a feature-specific operation.

L36: suspend fun flushQueue() { Function performs a feature-specific operation.

L40: suspend fun addParticipantToRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L44: suspend fun removeParticipantFromRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L48: suspend fun promoteParticipantToAdminOnRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L52: suspend fun demoteAdminOnRemote(chatId: String, userId: String) { Function performs a feature-specific operation.

L56: suspend fun createChatOnRemote(chat: ChatEntity) { Creates resources/documents.

L60: fun observeAllIncomingMessages(notificationHelper: NotificationHelper): Flow<Unit> { Returns observable/Flow for reactive updates.

L64: suspend fun syncUserChatsFromRemote() { Function performs a feature-specific operation.

L68: suspend fun findExistingP2PChat(uid: String, userId: String): ChatEntity? { Function performs a feature-specific operation.

L72: suspend fun fetchAndSyncMessages(chatId: String) { Function performs a feature-specific operation.

L76: suspend fun clearChatMessagesOnRemote(chatId: String) { Function performs a feature-specific operation.

L80: suspend fun deleteChatOnRemote(chatId: String) { Function performs a feature-specific operation.

L84: fun deleteChatAndMessagesGlobally(chatId: String) { Function performs a feature-specific operation.

L88: suspend fun getChatById(chatId: String): ChatEntity? { Getter/fetcher retrieves data.

L93: suspend fun sendSessionInvite(Function performs a feature-specific operation.

L103: suspend fun addSharedSessionToGroup(Function performs a feature-specific operation.

L113: fun observeSharedSessions(chatId: String): Flow<List<Map<String, Any>>> { Returns observable/Flow for reactive updates.

L117: suspend fun sendSessionPatch(chatId: String, sessionId: String, op: String, data: JSONObject) { Function performs a feature-specific operation.

L122: suspend fun sendSyncRequest(targetUserId: String, targetCollectionId: String) {
Function performs a feature-specific operation.

L126: fun observeIncomingRequests(): Flow<List<SyncRequest>> { Returns observable/Flow for reactive updates.

L130: suspend fun uploadQuestionBankZip(zipFile: File): Pair<String, String> { Function performs a feature-specific operation.

L134: suspend fun deleteQuestionBankZip(fileId: String) { Function performs a feature-specific operation.

L138: suspend fun shareCollectionToGroup(chatId: String, collectionMetadata: Map<String, Any>) { Function performs a feature-specific operation.

L142: suspend fun acknowledgeCollectionDownload(collectionId: String) { Function performs a feature-specific operation.

L146: suspend fun applyCollectionMicroUpdate(payload: String) { Function performs a feature-specific operation.

L150: suspend fun broadcastCollectionMicroUpdate(chatId: String, collectionId: String, diff: JSONObject) { Function performs a feature-specific operation.

L154: fun observeGroupLibrary(chatId: String): Flow<List<Map<String, Any>>> { Returns observable/Flow for reactive updates.

L158: suspend fun sendCollectionPatch(chatId: String, collectionId: String, op: String, data: JSONObject) { Function performs a feature-specific operation.

L162: suspend fun requestCollectionAccess(chatId: String, collectionId: String) { Function performs a feature-specific operation.

L166: fun observeAccessRequests(chatId: String): Flow<List<Map<String, Any>>> { Returns observable/Flow for reactive updates.

L170: suspend fun grantCollectionAccess(chatId: String, collectionId: String, requesterId: String) { Function performs a feature-specific operation.

L175: suspend fun reportSession(sessionId: String, reason: String) { Function performs a feature-specific operation.

L179: suspend fun reportQuestion(Function performs a feature-specific operation.

L188: suspend fun reportGroup(group: ChatEntity, reason: String) { Function performs a feature-specific operation.

L192: suspend fun reportUser(user: UserEntity, reason: String) { Function performs a feature-specific operation.

L196: suspend fun reportCollection(collection: StudyCollection, reason: String) { Function performs a feature-specific operation.

L200: suspend fun reportMessage(message: MessageEntity, reason: String) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/UniversalQueueManager.kt

L16: class UniversalQueueManager @Inject constructor(Class app component; see source for details.

L22: suspend fun flushUniversalQueue() { Function performs a feature-specific operation.

L56: private suspend fun processAction(action: OfflineActionEntity): Boolean { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/sync/ActionQueueDao.kt

L6: interface ActionQueueDao { Interface contract for implementations.

L8: suspend fun insertAction(action: OfflineActionEntity) Function performs a feature-specific operation.

L11: suspend fun getPendingActions(): List<OfflineActionEntity> Getter/fetcher retrieves data.

L14: suspend fun deleteAction(action: OfflineActionEntity) Function performs a feature-specific operation.

L17: suspend fun updateAction(action: OfflineActionEntity) Function performs a feature-specific operation.

```
L20: suspend fun clearAllActions()    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/data/sync/PatchEvent.kt

```
L6: data class PatchEvent(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/sync/SessionSyncRepository.kt

```
L27: class SessionSyncRepository @Inject constructor(    Repository    coordinates data sources  
for a feature.
```

```
L40: suspend fun sendSessionInvite(    Function    performs a feature-specific operation.
```

```
L62: suspend fun addSharedSessionToGroup(    Function    performs a feature-specific operation.
```

```
L115: fun observeSharedSessions(chatId: String): Flow<List<Map<String, Any>>> {    Returns  
observable/Flow for reactive updates.
```

```
L157: suspend fun sendSessionPatch(chatId: String, sessionId: String, op: String, data:  
JSONObject) {    Function    performs a feature-specific operation.
```

```
L174: suspend fun applySessionPatch(jsonString: String) {    Function    performs a  
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/data/sync/ReportSyncRepository.kt

```
L20: class ReportSyncRepository @Inject constructor(    Repository    coordinates data sources  
for a feature.
```

```
L29: suspend fun reportSession(sessionId: String, reason: String) {    Function    performs a  
feature-specific operation.
```

```

L52: suspend fun reportQuestion(    Function    performs a feature-specific operation.

L79: suspend fun reportGroup(group: ChatEntity, reason: String) {    Function    performs a
feature-specific operation.

L83: suspend fun reportUser(user: UserEntity, reason: String) {    Function    performs a
feature-specific operation.

L103: suspend fun reportCollection(collection: StudyCollection, reason: String) {    Function
performs a feature-specific operation.

L123: suspend fun reportMessage(message: MessageEntity, reason: String) {    Function
performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/collections/ExploreRepository.kt

```

L15: class ExploreRepository @Inject constructor(    Repository    coordinates data sources for
a feature.

L22: fun getCurrentUser(userId: String): Flow<UserEntity?> = userDao.getCurrentUser(userId)
Getter/fetcher    retrieves data.

L24: fun getStudyCollections(): Flow<List<StudyCollection>> =
collectionDao.getAllStudyCollections()    Getter/fetcher    retrieves data.

L26: fun getStudyCollectionsWithCount(): Flow<List<StudyCollectionWithCount>> =
Getter/fetcher    retrieves data.

L29: fun getStudyCollectionById(collectionId: String): Flow<StudyCollection?> =
collectionDao.getStudyCollectionById(collectionId)    Getter/fetcher    retrieves data.

L31: fun getSetsByStudyCollectionId(collectionId: String): Flow<List<QuestionSet>> =
collectionDao.getSetsByStudyCollectionId(collectionId)    Getter/fetcher    retrieves data.

L33: fun getLastSessionForStudyCollection(collectionId: String): Flow<StudySession?> =
sessionDao.getLastSessionForStudyCollection(collectionId)    Getter/fetcher    retrieves data.

L35: fun getQuestionsByStudyCollection(collection: String): Flow<List<Question>> =
Getter/fetcher    retrieves data.

L38: fun getQuestionsBySet(setId: String): Flow<List<Question>> =
collectionDao.getQuestionsForSet(setId)    Getter/fetcher    retrieves data.

L40: suspend fun getQuestionById(questionId: String): Question? =    Getter/fetcher    retrieves

```

data.

L43: suspend fun getSessionById(sessionId: String): StudySession? =
sessionDao.getSessionById(sessionId) Getter/fetcher retrieves data.

L45: suspend fun updateSession(session: StudySession) = sessionDao.updateSession(session)
Function performs a feature-specific operation.

L47: fun getOptionsForQuestion(questionId: String): Flow<List<QuestionOption>> =
Getter/fetcher retrieves data.

L50: fun getAnswerForQuestion(questionId: String): Flow<Answer?> = Getter/fetcher
retrieves data.

L53: suspend fun updateQuestion(question: Question) { Function performs a feature-specific
operation.

L57: suspend fun reportProblem(report: ProblemReport) { Function performs a
feature-specific operation.

L61: fun getAllSets(): Flow<List<QuestionSet>> = sessionDao.getAllSets() Getter/fetcher
retrieves data.

L63: fun getAllSessions(): Flow<List<StudySession>> = sessionDao.getAllSessions()
Getter/fetcher retrieves data.

L65: fun getPinnedQuestions(): Flow<List<Question>> = questionDao.getPinnedQuestions()
Getter/fetcher retrieves data.

L67: fun getQuestionCountByStudyCollection(collectionName: String): Flow<Int> =
Getter/fetcher retrieves data.

L70: suspend fun addQuestionToSet(setId: String, questionId: String) { Function performs a
feature-specific operation.

L76: suspend fun addQuestionToSession(sessionId: String, questionId: String) { Function
performs a feature-specific operation.

L88: suspend fun saveSet(set: QuestionSet) { Persists or updates data.

L92: suspend fun saveAnswer(answer: Answer) { Persists or updates data.

L96: suspend fun updateStudyCollection(collection: StudyCollection) { Function performs a
feature-specific operation.

L100: suspend fun getStudyCollectionByIdOnce(collectionId: String): StudyCollection? {
Getter/fetcher retrieves data.

L104: suspend fun getStudyCollectionByNameOnce(name: String): StudyCollection? {
Getter/fetcher retrieves data.

```
L108: suspend fun getSetIdForQuestion(questionId: String): String? {    Getter/fetcher
retrieves data.
```

```
L112: suspend fun deleteStudyCollection(collectionId: String) {    Function    performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/data/collections/ProblemReport.kt

```
L21: data class ProblemReport(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/CollectionVersionLedger.kt

```
L11: data class CollectionVersionLedgerEntity(    Data model    holds structured fields.
```

```
L18: interface CollectionVersionLedgerDao {    Interface    contract for implementations.
```

```
L20: suspend fun getLedgerForCollection(collectionId: String): CollectionVersionLedgerEntity?
    Getter/fetcher    retrieves data.
```

```
L23: suspend fun insertLedger(ledger: CollectionVersionLedgerEntity)    Function    performs a
feature-specific operation.
```

```
L26: suspend fun deleteLedger(collectionId: String)    Function    performs a feature-specific
operation.
```

app/src/main/java/com/algorithmx/q_base/data/collections/QuestionSet.kt

```
L23: data class QuestionSet(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/ImportRepository.kt

```
L20: class ImportRepository @Inject constructor(    Repository    coordinates data sources for a
feature.

L30: suspend fun recognizeTextFromImage(uri: Uri): Result<String> =
withContext(Dispatchers.IO) {    Function    performs a feature-specific operation.

L41: suspend fun extractTextFromPdf(uri: Uri): Result<String> = withContext(Dispatchers.IO) {
    Function    performs a feature-specific operation.

L71: fun cleanRecognizedText(rawText: String): String {    Function    performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/data/collections/QuestionWithContent.kt

```
L6: data class QuestionWithContent(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/Answer.kt

```
L10: data class Answer(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/CollectionWithSets.kt

```
L6: data class StudyCollectionWithSets(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/ProblemReportDao.kt

```
L7: interface ProblemReportDao {    Interface    contract for implementations.

L9: suspend fun insertReport(report: ProblemReport)    Function    performs a feature-specific
operation.

L12: fun getAllReports(): Flow<List<ProblemReport>>    Getter/fetcher    retrieves data.

L15: fun getReportsForQuestion(questionId: String): Flow<List<ProblemReport>>
Getter/fetcher    retrieves data.
```

app/src/main/java/com/algorithmx/q_base/data/collections/CollectionDao.kt

```
L7: interface CollectionDao {    Interface    contract for implementations.

L9: fun getAllStudyCollections(): Flow<List<StudyCollection>>    Getter/fetcher    retrieves
data.

L16: fun getAllStudyCollectionsWithCount(): Flow<List<StudyCollectionWithCount>>
Getter/fetcher    retrieves data.

L19: suspend fun getStudyCollectionCount(): Int    Getter/fetcher    retrieves data.

L22: fun getStudyCollectionById(collectionId: String): Flow<StudyCollection?>
Getter/fetcher    retrieves data.

L25: suspend fun getStudyCollectionByIdOnce(collectionId: String): StudyCollection?
Getter/fetcher    retrieves data.

L28: suspend fun getStudyCollectionByNameOnce(name: String): StudyCollection?
Getter/fetcher    retrieves data.

L31: fun getSetsByStudyCollectionId(collectionId: String): Flow<List<QuestionSet>>
Getter/fetcher    retrieves data.

L34: fun getSetsByStudyCollectionName(collectionName: String): Flow<List<QuestionSet>>
Getter/fetcher    retrieves data.

L41: fun getCategoriesByStudyCollectionId(collectionId: String): Flow<List<String>>
Getter/fetcher    retrieves data.
```

L48: fun getQuestionsForSet(setId: String): Flow<List<Question>> Getter/fetcher retrieves data.

L51: suspend fun getSetsByStudyCollectionIdOnce(collectionId: String): List<QuestionSet> Getter/fetcher retrieves data.

L58: suspend fun getQuestionsForSetOnce(setId: String): List<Question> Getter/fetcher retrieves data.

L61: suspend fun getCrossRefsForSetsBatch(setIds: List<String>): List<SetQuestionCrossRef> Getter/fetcher retrieves data.

L64: suspend fun insertStudyCollections(collections: List<StudyCollection>) Function performs a feature-specific operation.

L67: suspend fun updateStudyCollection(collection: StudyCollection) Function performs a feature-specific operation.

L70: suspend fun insertSets(sets: List<QuestionSet>) Function performs a feature-specific operation.

L73: suspend fun insertCrossRefs(refs: List<SetQuestionCrossRef>) Function performs a feature-specific operation.

L76: suspend fun updateStudyCollectionTimestamp(collectionId: String, timestamp: Long) Function performs a feature-specific operation.

L79: suspend fun deleteStudyCollectionById(collectionId: String) Function performs a feature-specific operation.

L82: suspend fun deleteAllStudyCollections() Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/data/collections/Question.kt

L18: data class Question(Data model holds structured fields.

app/src/main/java/com/algorithmx/q_base/data/collections/CollectionExport.kt

```
L5: data class CollectionExport(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/SetWithQuestions.kt

```
L7: data class SetWithQuestions(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/QuestionDao.kt

```
L7: interface QuestionDao {    Interface    contract for implementations.
```

```
L9: fun getQuestionsByStudyCollection(collection: String): Flow<List<Question>>
Getter/fetcher    retrieves data.
```

```
L12: fun getQuestionsByCategory(category: String): Flow<List<Question>>    Getter/fetcher
retrieves data.
```

```
L15: fun getQuestionsByTag(tag: String): Flow<List<Question>>    Getter/fetcher    retrieves
data.
```

```
L18: fun getQuestionsByCategoryAndTag(category: String, tag: String): Flow<List<Question>>
Getter/fetcher    retrieves data.
```

```
L21: fun getOptionsForQuestion(questionId: String): Flow<List<QuestionOption>>
Getter/fetcher    retrieves data.
```

```
L24: fun getAnswerForQuestion(questionId: String): Flow<Answer?>    Getter/fetcher    retrieves
data.
```

```
L27: suspend fun getOptionsForQuestionOnce(questionId: String): List<QuestionOption>
Getter/fetcher    retrieves data.
```

```
L30: suspend fun getAnswerForQuestionOnce(questionId: String): Answer?    Getter/fetcher
retrieves data.
```

```
L33: suspend fun getQuestionById(questionId: String): Question?    Getter/fetcher    retrieves
data.
```

L36: suspend fun updateQuestion(question: Question) Function performs a feature-specific operation.

L39: suspend fun insertQuestions(questions: List<Question>) Function performs a feature-specific operation.

L42: suspend fun insertOptions(options: List<QuestionOption>) Function performs a feature-specific operation.

L45: suspend fun insertAnswers(answers: List<Answer>) Function performs a feature-specific operation.

L48: suspend fun insertSet(set: QuestionSet) Function performs a feature-specific operation.

L51: suspend fun insertQuestion(question: Question) Function performs a feature-specific operation.

L54: suspend fun insertOption(option: QuestionOption) Function performs a feature-specific operation.

L57: suspend fun insertAnswer(answer: Answer) Function performs a feature-specific operation.

L60: suspend fun insertSetQuestionCrossRef(crossRef: SetQuestionCrossRef) Function performs a feature-specific operation.

L63: fun getQuestionCountFlow(): Flow<Int> Getter/fetcher retrieves data.

L71: fun getUserCreatedQuestionCount(): Flow<Int> Getter/fetcher retrieves data.

L79: fun getSharedQuestionCount(): Flow<Int> Getter/fetcher retrieves data.

L82: suspend fun getQuestionCount(): Int Getter/fetcher retrieves data.

L85: fun getPinnedQuestions(): Flow<List<Question>> Getter/fetcher retrieves data.

L89: suspend fun getSetWithContent(setId: String): SetWithQuestions? Getter/fetcher retrieves data.

L92: fun getQuestionCountByStudyCollection(collectionName: String): Flow<Int> Getter/fetcher retrieves data.

L95: suspend fun getSetIdForQuestion(questionId: String): String? Getter/fetcher retrieves data.

L98: suspend fun deleteOptionsForQuestion(questionId: String) Function performs a feature-specific operation.

L101: suspend fun deleteSetById(setId: String) Function performs a feature-specific operation.

```

L104: suspend fun deleteSetsByIds(setIds: List<String>)    Function performs a
feature-specific operation.

L107: suspend fun deleteCrossRefsForSets(setIds: List<String>)    Function performs a
feature-specific operation.

L110: suspend fun removeQuestionFromSet(setId: String, questionId: String)    Function
performs a feature-specific operation.

L113: suspend fun deleteQuestionById(questionId: String)    Function performs a
feature-specific operation.

L116: suspend fun deleteAllQuestions()    Function performs a feature-specific operation.

L119: suspend fun deleteAllOptions()    Function performs a feature-specific operation.

L122: suspend fun deleteAllAnswers()    Function performs a feature-specific operation.

L125: suspend fun deleteAllCrossRefs()    Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/data/collections/MockExport.kt

```

L5: data class QuestionExport(    Data model holds structured fields.

L12: data class MockExport(    Data model holds structured fields.

```

app/src/main/java/com/algorithmx/q_base/data/collections/QuestionOption.kt

```

L23: data class QuestionOption(    Data model holds structured fields.

```

app/src/main/java/com/algorithmx/q_base/data/collections/StudyCollection.kt

```
L10: data class StudyCollection(    Data model    holds structured fields.
```

```
L37: data class StudyCollectionWithCount(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/data/collections/SetQuestionCrossRef.kt

```
L32: data class SetQuestionCrossRef(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/util/NotificationHelper.kt

```
L21: class NotificationHelper @Inject constructor(    Class    app component; see source for details.
```

```
L30: fun createNotificationChannels() {    Creates resources/documents.
```

```
L55: fun showMessageNotification(chatId: String, senderName: String, message: String) {
Function    performs a feature-specific operation.
```

```
L87: fun showSessionNotification(sessionId: String, title: String, description: String) {
Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/util/NetworkMonitor.kt

```
L21: class NetworkMonitor @Inject constructor(    Class    app component; see source for details.
```

```
L32: fun currentConnectivity(): Boolean {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/navigation/Screen.kt

L7: sealed class Screen : NavKey { Sealed hierarchy closed set of subclasses.

app/src/main/java/com/algorithmx/q_base/ui/navigation/Navigator.kt

L8: class Navigator(val state: NavigationState) { Class app component; see source for details.

L9: private fun normalizeRoute(route: NavKey): NavKey = when (route) { Function performs a feature-specific operation.

L14: private fun findTopLevelMatch(route: NavKey): NavKey? = Function performs a feature-specific operation.

L19: private fun resetStack(root: NavKey, stack: androidx.navigation3.runtime.NavBackStack<NavKey>) { Function performs a feature-specific operation.

L24: fun navigate(route: NavKey) { Function performs a feature-specific operation.

L56: fun resetTo(route: NavKey) { Function performs a feature-specific operation.

L78: fun goBack() { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/ui/navigation/NavigationState.kt

L27: fun rememberNavigationState(Function performs a feature-specific operation.

L83: class NavigationState(Class app component; see source for details.

L98: fun NavigationState.toEntries(Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/ui/navigation/AppEntryProvider.kt

```
L16: fun rememberAppEntryProvider(navigator: Navigator) = remember(navigator) {    Function
performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/navigation/AppEntryWrappers.kt

```
L20: fun ExplorePagerWrapper(key: Screen.ExplorePager, navigator: Navigator) {    Function
performs a feature-specific operation.
```

```
L81: fun CollectionOverviewWrapper(key: Screen.CollectionOverview, navigator: Navigator) {
Function    performs a feature-specific operation.
```

```
L123: fun PinnedQuestionsWrapper(navigator: Navigator) {    Function    performs a
feature-specific operation.
```

```
L147: fun ExploreSetWrapper(key: Screen.ExploreSet, navigator: Navigator) {    Function
performs a feature-specific operation.
```

```
L193: fun SessionsWrapper(navigator: Navigator) {    Function    performs a feature-specific
operation.
```

```
L220: fun NewSessionWizardWrapper(navigator: Navigator) {    Function    performs a
feature-specific operation.
```

```
L238: fun ActiveSessionWrapper(key: Screen.ActiveSession, navigator: Navigator) {    Function
performs a feature-specific operation.
```

```
L253: fun SessionResultsWrapper(key: Screen.SessionResults, navigator: Navigator) {
Function    performs a feature-specific operation.
```

```
L267: fun ChatDetailWrapper(key: Screen.ChatDetail, navigator: Navigator) {    Function
performs a feature-specific operation.
```

```
L294: fun ContactOverviewWrapper(key: Screen.ContactOverview, navigator: Navigator) {
Function    performs a feature-specific operation.
```

```
L308: fun GroupOverviewWrapper(key: Screen.GroupOverview, navigator: Navigator) {    Function
performs a feature-specific operation.
```

```
L322: fun ProfileWrapper(navigator: Navigator) {    Function    performs a feature-specific
operation.
```

app/src/main/java/com/algorithmx/q_base/ui/explore/ExploreViewModel.kt

```

L20: data class ExploreQuestionState(    Data model holds structured fields.

L32: class ExploreViewModel @Inject constructor(    ViewModel manages UI state and
interactions.

L112: fun resetQuestionStates() {    Function performs a feature-specific operation.

L141: fun loadQuestionsByStudyCollection(collectionId: String) {    Function performs a
feature-specific operation.

L156: fun loadQuestionsBySet(setId: String) {    Function performs a feature-specific
operation.

L166: private suspend fun checkIsEditable(collection: StudyCollection?): Boolean {    Function
performs a feature-specific operation.

L175: fun loadPinnedQuestions() {    Function performs a feature-specific operation.

L191: fun updateCollectionAdminOnly(collectionId: String, isAdminOnly: Boolean) {    Function
performs a feature-specific operation.

L213: fun loadCollectionOverview(collectionId: String) {    Function performs a
feature-specific operation.

L246: fun updateSessionProgress(sessionId: String, index: Int) {    Function performs a
feature-specific operation.

L254: fun loadQuestionDetails(index: Int) {    Function performs a feature-specific
operation.

L272: fun selectOption(index: Int, optionLetter: String) {    Function performs a
feature-specific operation.

L309: fun revealAnswer(index: Int) {    Function performs a feature-specific operation.

L317: fun togglePin(index: Int) {    Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/explore/ExploreViewModelSetExtensions.kt

```

L11: fun ExploreViewModel.toggleSetSelection(setId: String) {    Function performs a
feature-specific operation.

L22: fun ExploreViewModel.clearSelection() {    Function performs a feature-specific
operation.

L27: fun ExploreViewModel.deleteCollectionSet(setId: String) {    Function performs a
feature-specific operation.

L34: suspend fun ExploreViewModel.getSetIdForQuestion(questionId: String): String? {
Function performs a feature-specific operation.

L38: fun ExploreViewModel.deleteSelectedSets() {    Function performs a feature-specific
operation.

L50: fun ExploreViewModel.addQuestionToSet(index: Int, setId: String) {    Function performs
a feature-specific operation.

L72: fun ExploreViewModel.addQuestionToSession(index: Int, sessionId: String) {    Function
performs a feature-specific operation.

L94: fun ExploreViewModel.createSet(title: String, description: String, collectionId: String)
{    Function performs a feature-specific operation.

L110: fun ExploreViewModel.deleteQuestion(index: Int) {    Function performs a
feature-specific operation.

L137: fun ExploreViewModel.deleteQuestionFromSet(index: Int, setId: String) {    Function
performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/explore/ExploreViewModel AiExtensions.kt

```

L9: fun ExploreViewModel.askAi(index: Int, mode: String = "EXPLAIN") {    Function performs a
feature-specific operation.

L37: fun ExploreViewModel.askAiAboutCollection(collection: StudyCollection) {    Function
performs a feature-specific operation.

L56: fun ExploreViewModel.clearCollectionAiResponse() {    Function performs a
feature-specific operation.

L60: fun ExploreViewModel.clearAiResponse(index: Int) {    Function performs a
feature-specific operation.

```

```
L66: fun ExploreViewModel.saveAiResponseToQuestion(index: Int) {    Function performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/explore/PinnedQuestionsScreen.kt

```
L29: fun PinnedQuestionsScreen(    Function performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/explore/ExploreScreens.kt

```
L45: fun ExploreQuestionPagerScreen(    Function performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/explore/UnifiedExploreScreen.kt

```
L37: fun UnifiedExploreScreen(    Function performs a feature-specific operation.

L172: fun MasterCollectionListItem(    Function performs a feature-specific operation.

L256: fun EmptyLibraryView(modifier: Modifier = Modifier) {    Function performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/explore/ExploreViewModelReportExtensions.kt

```
L10: fun ExploreViewModel.reportCollectionToGroup(collection: StudyCollection, reason:
String) {    Function performs a feature-specific operation.

L32: fun ExploreViewModel.reportProblem(index: Int, explanation: String) {    Function
performs a feature-specific operation.
```

```

L62: fun ExploreViewModel.reportCollection(collection: StudyCollection, reason: String) {
Function performs a feature-specific operation.

L79: fun ExploreViewModel.deleteStudyCollection(collectionId: String) { Function performs
a feature-specific operation.

L92: fun ExploreViewModel.reportSet(setId: String, reason: String) { Function performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/explore/CollectionOverviewScreen.kt

```

L39: fun CollectionOverviewScreen( Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/explore/CollectionOverviewComponents.kt

```

L21: fun StatCard( Function performs a feature-specific operation.

L44: fun SetItem( Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/state/AppAccessState.kt

```

L3: sealed interface AppAccessState { Interface contract for implementations.

L4: data object RestoringSession : AppAccessState Singleton object shared utilities/DI.

L5: data object OnlineReady : AppAccessState Singleton object shared utilities/DI.

L6: data object SignedInOffline : AppAccessState Singleton object shared utilities/DI.

L7: data object GuestOnline : AppAccessState Singleton object shared utilities/DI.

L8: data object OfflineGuest : AppAccessState Singleton object shared utilities/DI.

```

app/src/main/java/com/algorithmx/q_base/ui/sessions/NewSessionWizardScreen.kt

```
L26: fun NewSessionWizardScreen(    Function    performs a feature-specific operation.  
  
L140: fun EmptyCollectionsView() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/sessions/SessionsScreens.kt

```
L51: fun SessionsListScreen(    Function    performs a feature-specific operation.  
  
L176: fun AnimatedSessionItem(index: Int, content: @Composable () -> Unit) {    Function  
performs a feature-specific operation.  
  
L192: fun CategoryChip(name: String) {    Function    performs a feature-specific operation.  
  
L220: fun SessionListItemExpressive(    Function    performs a feature-specific operation.  
  
L319: fun EmptySessionsView() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/sessions/SessionWizardSteps.kt

```
L27: fun CategoryStep(    Function    performs a feature-specific operation.  
  
L75: fun QuestionSelectionStep(    Function    performs a feature-specific operation.  
  
L182: fun ConfigurationStep(    Function    performs a feature-specific operation.  
  
L284: fun ConfigToggle(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/sessions/ActiveSessionViewModel.kt

```

L25: sealed class SessionNavEvent {    Sealed hierarchy    closed set of subclasses.

L26: data class NavigateToResults(val sessionId: String) : SessionNavEvent()    Data model
holds structured fields.

L29: data class NavigatorDot(    Data model    holds structured fields.

L36: class ActiveSessionViewModel @Inject constructor(    ViewModel    manages UI state and
interactions.

L116: fun setSessionId(id: String, chatId: String? = null) {    Persists or updates data.

L128: fun getSessionId(): String = _sessionId    Getter/fetcher    retrieves data.

L130: private fun loadSessionData() {    Function    performs a feature-specific operation.

L156: private fun startTimer() {    Function    performs a feature-specific operation.

L184: private fun loadAttempts() {    Function    performs a feature-specific operation.

L195: fun navigateToQuestion(index: Int) {    Function    performs a feature-specific operation.

L205: private fun loadQuestion(questionId: String) {    Function    performs a feature-specific
operation.

L216: fun onAnswerSelected(optionLetter: String) {    Function    performs a feature-specific
operation.

L257: fun toggleFlag() {    Function    performs a feature-specific operation.

L264: private fun updateAttempt(attempt: SessionAttempt) {    Function    performs a
feature-specific operation.

L290: fun askAi(mode: String = "EXPLAIN") {    Function    performs a feature-specific
operation.

L309: fun clearAiResponse() {    Function    performs a feature-specific operation.

L313: fun saveAiResponseToQuestion() {    Persists or updates data.

L331: fun submitSession() {    Function    performs a feature-specific operation.

L366: fun reportSession(reason: String) {    Function    performs a feature-specific operation.

L380: fun reportQuestion(reason: String) {    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/sessions/ActiveSessionScreen.kt

```
L56: fun ActiveSessionScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/sessions/SessionsViewModel.kt

```
L20: class SessionsViewModel @Inject constructor(    ViewModel    manages UI state and interactions.

L96: fun onSearchQueryChange(query: String) {    Function    performs a feature-specific operation.

L100: fun setWizardStep(step: Int) {    Persists or updates data.

L104: fun selectCollection(name: String) {    Function    performs a feature-specific operation.

L114: fun toggleQuestionSelection(id: String) {    Function    performs a feature-specific operation.

L121: fun selectAllQuestions() {    Function    performs a feature-specific operation.

L125: fun deselectAllQuestions() {    Function    performs a feature-specific operation.

L129: fun selectRandomQuestions(count: Int) {    Function    performs a feature-specific operation.

L140: fun setOrder(order: String) { _sessionOrder.value = order }    Persists or updates data.

L141: fun setTimingType(type: String) { _timingType.value = type }    Persists or updates data.

L142: fun setTimeLimit(seconds: Int) { _timeLimitSeconds.value = seconds }    Persists or updates data.

L143: fun setSessionIsAdminOnly(value: Boolean) { _sessionIsAdminOnly.value = value }    Persists or updates data.
```

```

L145: fun launchSession(title: String) {    Function    performs a feature-specific operation.

L163: fun resetWizard() {    Function    performs a feature-specific operation.

L174: fun toggleSessionSelection(sessionId: String) {    Function    performs a feature-specific
operation.

L185: fun clearSessionSelection() {    Function    performs a feature-specific operation.

L190: fun deleteSelectedSessions() {    Function    performs a feature-specific operation.

L200: fun reportSession(sessionId: String, reason: String) {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/sessions/SessionResultsViewModel.kt

```

L20: data class ReviewState(    Data model    holds structured fields.

L27: sealed class ResultsUiState {    Sealed hierarchy    closed set of subclasses.

L28: object Loading : ResultsUiState()    Singleton object    shared utilities/DI.

L29: data class Success(val session: StudySession?, val attempts: List<SessionAttempt>, val
score: Float) : ResultsUiState()    Data model    holds structured fields.

L33: class SessionResultsViewModel @Inject constructor(    ViewModel    manages UI state and
interactions.

L66: fun initSession(id: String) {    Function    performs a feature-specific operation.

L72: private fun loadResults() {    Function    performs a feature-specific operation.

L104: fun updateSessionAdminOnly(sessionId: String, isAdminOnly: Boolean) {    Function
performs a feature-specific operation.

L128: fun selectQuestionForReview(attempt: SessionAttempt) {    Function    performs a
feature-specific operation.

L143: fun clearReview() {    Function    performs a feature-specific operation.

L147: fun reportCurrentSession(reason: String) {    Function    performs a feature-specific
operation.

```

app/src/main/java/com/algorithmx/q_base/ui/sessions/SessionResultsScreen.kt

```
L39: fun SessionResultsScreen(    Function    performs a feature-specific operation.

L162: fun ResultsContent(    Function    performs a feature-specific operation.

L334: fun AnimatedAttemptDot(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/sessions/components/MasterNavigator.kt

```
L21: fun MasterNavigator(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/settings/AiBrainManagerScreen.kt

```
L22: fun AiBrainManagerScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/settings/SecureBackupViewModel.kt

```
L14: data class SecureBackupState(    Data model    holds structured fields.

L22: class SecureBackupViewModel @Inject constructor(    ViewModel    manages UI state and
interactions.

L34: fun checkBackupStatus() {    Function    performs a feature-specific operation.

L43: fun setupBackup(passphrase: String) {    Persists or updates data.

L64: fun restoreBackup(passphrase: String) {    Function    performs a feature-specific
```

operation.

L80: fun clearState() { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/ui/settings/SettingsScreen.kt

L34: fun SettingsScreen(Function performs a feature-specific operation.

L58: fun SettingsContent(Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/ui/settings/ProfileViewModel.kt

L23: data class UserStats(Data model holds structured fields.

L31: class ProfileViewModel @Inject constructor(ViewModel manages UI state and interactions.

L82: private fun loadUser() { Function performs a feature-specific operation.

L96: private fun loadBackupStatus() { Function performs a feature-specific operation.

L110: private fun loadStats() { Function performs a feature-specific operation.

L130: private suspend fun tryUpdateProfile(updatedProfile: UserProfile) { Function performs a feature-specific operation.

L142: fun updateDisplayName(newName: String) { Function performs a feature-specific operation.

L160: fun updateIntro(newIntro: String) { Function performs a feature-specific operation.

L178: fun updateProfilePictureUrl(url: String) { Function performs a feature-specific operation.

L196: fun togglePhotoVisibility(isVisible: Boolean) { Function performs a feature-specific operation.

L214: fun signOut(clearCollections: Boolean, onComplete: () -> Unit) { Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/ui/settings/ProfileComponents.kt

```
L40: data class AvatarTemplate(    Data model    holds structured fields.

L59: fun Modifier.bounceClick(onClick: () -> Unit = {}) = composed {    Function    performs a
feature-specific operation.

L91: fun ProfileParallaxHeader(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/settings/SettingsViewModel.kt

```
L25: class SettingsViewModel @Inject constructor(    ViewModel    manages UI state and
interactions.

L44: fun updateModel(newModel: String) {    Function    performs a feature-specific operation.

L56: fun updateSystemInstruction(instruction: String) {    Function    performs a
feature-specific operation.

L67: fun saveTaskConfig(task: BrainTask, config: TaskConfig) {    Persists or updates data.

L73: fun updateNotifications(enabled: Boolean) {    Function    performs a feature-specific
operation.

L79: fun updateTheme(mode: String) {    Function    performs a feature-specific operation.

L85: fun clearAllData(onComplete: () -> Unit) {    Function    performs a feature-specific
operation.

L94: private fun calculateDbSize(): Double {    Function    performs a feature-specific
operation.
```

app/src/main/java/com/algorithmx/q_base/ui/settings/AppThemeScreen

.kt

```

L26: fun AppThemeScreen(    Function    performs a feature-specific operation.

L85: data class ThemeOption(val id: String, val title: String, val subtitle: String)    Data
model    holds structured fields.

L88: fun ThemeCard(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/SecureBackupDialog.kt

```

L16: fun SecureBackupDialog(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/ProfileScreen.kt

```

L35: fun ProfileScreen(    Function    performs a feature-specific operation.

L74: fun ProfileContent(    Function    performs a feature-specific operation.

L290: fun ProfileContentPreview() {    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/components/ProfileAvatar.kt

```

L29: fun ProfileAvatar(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/components/ProfileRowComponents.kt

```

L19: fun ProfileCardSection(    Function    performs a feature-specific operation.

L54: fun ProfilePropertyRow(    Function    performs a feature-specific operation.

L113: fun ProfilePropertyToggleRow(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/components/SettingsComponents.kt

```

L18: fun SettingsSection(    Function    performs a feature-specific operation.

L31: fun SettingsSectionHeader(title: String) {    Function    performs a feature-specific
operation.

L44: fun SettingsToggleCard(    Function    performs a feature-specific operation.

L81: fun SettingsCard(    Function    performs a feature-specific operation.

L117: fun UsageStatsCard(requests: Int, tokens: Int) {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/settings/components/ProfileStatsComponents.kt

```

L28: fun ProfileAchievementsProperty(stats: UserStats) {    Function    performs a
feature-specific operation.

L63: fun BadgeBadge(    Function    performs a feature-specific operation.

L134: fun StatCard(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/theme/Theme.kt

```

L98: fun QbaseTheme(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/ContactOverviewScreen.kt

```
L34: fun ContactOverviewScreen(    Function    performs a feature-specific operation.  
  
L196: fun ActionButton(    Function    performs a feature-specific operation.  
  
L221: fun SettingsItem(    Function    performs a feature-specific operation.  
  
L260: fun ReportDialog(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatDetailScreen.kt

```
L27: fun ChatDetailScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/GroupOverviewScreen.kt

```
L35: fun GroupOverviewScreen(    Function    performs a feature-specific operation.  
  
L334: fun ParticipantItem(user: UserEntity, isAdmin: Boolean, onClick: () -> Unit = {}) {  
    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/NewChatScreen.kt

```
L10: fun NewChatScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatViewModelSharingExtensions.kt

```

L10: fun ChatViewModel.addSharedCollection(jsonPayload: String) {    Function    performs a
feature-specific operation.

L27: fun ChatViewModel.importSharedCollection(payload: String) {    Function    performs a
feature-specific operation.

L80: fun ChatViewModel.shareCollection(chatId: String, collectionId: String) {    Function
performs a feature-specific operation.

L132: fun ChatViewModel.resendCollection(collectionId: String) {    Function    performs a
feature-specific operation.

L176: fun ChatViewModel.shareSession(chatId: String, sessionId: String) {    Function
performs a feature-specific operation.

L217: fun ChatViewModel.joinSession(sessionIdOrPayload: String, onSessionImported: (String)
-> Unit) {    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatListScreen.kt

```

L44: fun ChatListScreen(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/BlockedListScreen.kt

```

L26: fun BlockedListScreen(    Function    performs a feature-specific operation.

L71: fun BlockedChatItem(    Function    performs a feature-specific operation.

L129: fun EmptyBlockedView(modifier: Modifier = Modifier) {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/AddParticipantDialog

.kt

```
L23: fun AddParticipantDialog(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/NewGroupScreen.kt

```
L29: fun NewGroupScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatModels.kt

```
L10: data class ChatUiModel(    Data model    holds structured fields.
```

```
L20: data class ChatListState(    Data model    holds structured fields.
```

```
L29: data class ChatDetailState(    Data model    holds structured fields.
```

```
L40: sealed class ChatNavEvent {    Sealed hierarchy    closed set of subclasses.
```

```
L41: data class NavigateToChatDetail(val chatId: String) : ChatNavEvent()    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatViewModel.kt

```
L34: class ChatViewModel @Inject constructor(    ViewModel    manages UI state and interactions.
```

```
L72: fun canSendToChat(chatId: String): Flow<Boolean> {    Returns observable/Flow for reactive updates.
```

```
L95: fun requestAccess(collectionId: String) {    Function    performs a feature-specific operation.
```

```
L107: fun grantAccess(collectionId: String, requesterId: String) {    Function    performs a feature-specific operation.
```

```
L137: fun toggleLibraryMode(enabled: Boolean) {    Function    performs a feature-specific operation.
```

```
L176: fun toggleChatSelection(chatId: String) {    Function    performs a feature-specific operation.
```

```
L188: fun clearSelection() {    Function    performs a feature-specific operation.
```

```
L193: fun deleteSelectedChats() {    Function    performs a feature-specific operation.
```

```
L348: fun syncChatsFromRemote() {    Function    performs a feature-specific operation.
```

```
L361: fun setChatId(chatId: String) {    Persists or updates data.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ContactSelector.kt

```
L38: fun ContactSelector(    Function    performs a feature-specific operation.
```

```
L301: fun UserItem(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatViewModelAdminExtensions.kt

```
L9: fun ChatViewModel.addParticipant(chatId: String, userId: String) {    Function    performs a feature-specific operation.
```

```
L48: fun ChatViewModel.removeParticipant(chatId: String, userId: String) {    Function    performs a feature-specific operation.
```

```
L86: fun ChatViewModel.promoteParticipantToAdmin(chatId: String, userId: String) {    Function    performs a feature-specific operation.
```

```
L109: fun ChatViewModel.demoteAdmin(chatId: String, userId: String) {    Function    performs a feature-specific operation.
```

```
L136: fun ChatViewModel.leaveGroup(chatId: String) {    Function    performs a feature-specific operation.
```

```
L156: fun ChatViewModel.reportGroup(chatId: String, reason: String) {    Function    performs a feature-specific operation.
```

```

L170: fun ChatViewModel.reportUser(userId: String, reason: String) {    Function performs a
feature-specific operation.

L183: fun ChatViewModel.reportMessage(message: MessageEntity, reason: String) {    Function
performs a feature-specific operation.

L194: fun ChatViewModel.toggleMute(chatId: String, isMuted: Boolean) {    Function performs a
feature-specific operation.

L203: fun ChatViewModel.toggleBlock(chatId: String, isBlocked: Boolean) {    Function
performs a feature-specific operation.

L212: fun ChatViewModel.deleteChat(chatId: String) {    Function performs a feature-specific
operation.

L216: fun ChatViewModel.clearChatMessages(chatId: String) {    Function performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/ChatViewModelMessageExtensions.kt

```

L9: fun ChatViewModel.startNewChat(userId: String, userName: String) {    Function performs a
feature-specific operation.

L54: fun ChatViewModel.startNewGroup(participantIds: List<String>, groupName: String) {
Function performs a feature-specific operation.

L77: fun ChatViewModel.startAiChat() {    Function performs a feature-specific operation.

L81: fun ChatViewModel.sendMessage(chatId: String, text: String, type: String = "TEXT",
senderId: String = currentUserId) {    Function performs a feature-specific operation.

L130: private fun ChatViewModel.handleAiChatResponse(chatId: String, userMessage: String) {
Function performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/chat/ContactSelectorViewModel.kt

```

L13: data class ContactSelectorState(    Data model holds structured fields.

```

```
L21: class ContactSelectorViewModel @Inject constructor(    ViewModel    manages UI state and
interactions.
```

```
L33: private fun loadLocalContacts() {    Function    performs a feature-specific operation.
```

```
L41: fun searchByFriendCode(code: String, excludedUserId: String? = null) {    Function
performs a feature-specific operation.
```

```
L93: fun clearSearchResult() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/ChatItem.kt

```
L24: fun AnimatedChatItem(    Function    performs a feature-specific operation.
```

```
L49: fun ChatItem(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/MessageBubble.kt

```
L43: fun AnimatedMessageItem(    Function    performs a feature-specific operation.
```

```
L82: fun MessageBubble(    Function    performs a feature-specific operation.
```

```
L177: private fun AIHeader(isAiLoading: Boolean) {    Function    performs a feature-specific
operation.
```

```
L210: private fun SenderAvatar(avatarUrl: String?, onClick: () -> Unit) {    Function
performs a feature-specific operation.
```

```
L240: private fun SenderNameLabel(name: String, onClick: () -> Unit) {    Function    performs a
feature-specific operation.
```

```
L253: private fun MessageContent(    Function    performs a feature-specific operation.
```

```
L324: fun MessageBubblePreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/EmptyConnectView.kt

```
L15: fun EmptyConnectView(modifier: Modifier = Modifier) {    Function    performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/ChatDetailDialogs.kt

```
L23: fun SessionPickerSheet(    Function    performs a feature-specific operation.

L85: fun CollectionPickerSheet(    Function    performs a feature-specific operation.

L152: fun ChatDetailConfirmDialogs(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/SharedLibraryView.kt

```
L27: fun SharedLibraryView(    Function    performs a feature-specific operation.

L94: fun CollectionsTabContent(    Function    performs a feature-specific operation.

L151: fun SessionsTabContent(    Function    performs a feature-specific operation.

L183: fun LibraryEmptyState(message: String) {    Function    performs a feature-specific
operation.

L199: fun LibraryHeader(title: String, description: String) {    Function    performs a
feature-specific operation.

L207: fun SharedSessionCard(title: String, timestamp: Long, onJoin: () -> Unit) {    Function
performs a feature-specific operation.

L231: fun PendingRequestsSection(accessRequests: List<Map<String, Any>>, onGrantAccess:
(String, String) -> Unit) {    Function    performs a feature-specific operation.

L245: fun SharedCollectionCard(    Function    performs a feature-specific operation.
```

```
L375: fun AccessRequestItem(    Function    performs a feature-specific operation.

L412: fun SharedLibraryViewPreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/ChatDetailHelpers.kt

```
L9: fun accessStateLabel(state: AppAccessState): String = when (state) {    Function    performs
a feature-specific operation.

L17: fun formatDateRelatively(timestamp: Long): String {    Function    performs a
feature-specific operation.

L28: private fun isSameDay(cal1: Calendar, cal2: Calendar): Boolean {    Function    performs a
feature-specific operation.

L33: private fun isYesterday(now: Calendar, date: Calendar): Boolean {    Function    performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/GuestConnectView.kt

```
L17: fun GuestConnectView(modifier: Modifier = Modifier) {    Function    performs a
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/ChatDetailComponents.kt

```
L30: fun DateHeader(date: String) {    Function    performs a feature-specific operation.

L53: fun SystemMessageItem(text: String) {    Function    performs a feature-specific operation.

L76: fun ChatDetailTopBar(    Function    performs a feature-specific operation.
```

```
L213: fun ChatDetailBottomBar(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/chat/components/SpecialBubbleContents.kt

```
L23: fun CollectionBubbleContent(    Function    performs a feature-specific operation.
```

```
L64: fun SessionBubbleContent(payload: String, onJoin: (String) -> Unit, isMine: Boolean) {
Function    performs a feature-specific operation.
```

```
L94: fun FileTransferBubbleContent(    Function    performs a feature-specific operation.
```

```
L144: fun DecryptionErrorContent(status: String, isMine: Boolean, onDeleteChat: () -> Unit) {
    Function    performs a feature-specific operation.
```

```
L183: fun MessageTimestampAndStatus(timeString: String, isMine: Boolean, status: String) {
Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/auth/AuthViewModel.kt

```
L15: data class AuthState(    Data model    holds structured fields.
```

```
L24: class AuthViewModel @Inject constructor(    ViewModel    manages UI state and interactions.
```

```
L32: fun signIn(email: String, pass: String) {    Function    performs a feature-specific
operation.
```

```
L54: fun signUp(email: String, pass: String, username: String, photoUrl: String? = null) {
Function    performs a feature-specific operation.
```

```
L76: fun signInWithGoogle(activity: androidx.activity.ComponentActivity) {    Function
performs a feature-specific operation.
```

```
L93: fun onGoogleSignInSuccess(user: AppwriteUser) {    Function    performs a feature-specific
operation.
```

```
L105: fun clearError() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/auth/SignupScreen.kt

```
L38: fun SignupScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/auth/LoginScreen.kt

```
L35: fun LoginScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/ai/AiGenerationScreen.kt

```
L20: fun AiGenerationScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/ai/AiViewModel.kt

```
L16: sealed class AiUiState {    Sealed hierarchy    closed set of subclasses.

L17: object Idle : AiUiState()    Singleton object    shared utilities/DI.

L18: object Loading : AiUiState()    Singleton object    shared utilities/DI.

L19: data class Success(val message: String) : AiUiState()    Data model    holds structured
fields.

L20: data class Error(val message: String) : AiUiState()    Data model    holds structured
fields.

L24: class AiViewModel @Inject constructor(    ViewModel    manages UI state and interactions.

L37: fun generateCollection(    Generates content (often AI).

L72: fun promoteResponse(targetCollectionId: String? = null, targetCollectionName: String? =
null) {    Function    performs a feature-specific operation.

L86: fun resetState() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/home/HomeScreen.kt

```
L40: fun HomeScreen(    Function    performs a feature-specific operation.

L195: fun QuickActionCard(    Function    performs a feature-specific operation.

L230: fun HomeCategoryCard(    Function    performs a feature-specific operation.

L303: fun AnimatedHomeItem(    Function    performs a feature-specific operation.

L324: fun PinnedQuestionItem(    Function    performs a feature-specific operation.

L371: fun EmptyHomeView(onNavigate: () -> Unit) {    Function    performs a feature-specific
operation.
```

app/src/main/java/com/algorithmx/q_base/ui/home/HomeViewModel.kt

```
L20: class HomeViewModel @Inject constructor(    ViewModel    manages UI state and interactions.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Extraction WizardSecondScreen.kt

```
L24: fun ExtractionWizardSecondScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/QuestionEditorScreen.kt

```
L24: fun QuestionEditorScreen(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Common WizardThirdScreen.kt

```
L26: fun WaitingView(message: String) {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/ManualBuilderViewModel.kt

```
L18: class ManualBuilderViewModel @Inject constructor(    ViewModel    manages UI state and interactions.
```

```
L31: fun initialize(targetId: String?, name: String? = null) {    Function    performs a feature-specific operation.
```

```
L61: private suspend fun createNewCollectionAndSet(name: String? = null) {    Creates resources/documents.
```

```
L77: private suspend fun createNewSet(collectionId: String, title: String) {    Creates resources/documents.
```

```
L90: private fun observeQuestions(setId: String) {    Returns observable/Flow for reactive updates.
```

```
L96: fun getTargetSetId(): String? = _targetSetId.value    Getter/fetcher    retrieves data.
```

```
L98: private fun getCurrentDate(): String {    Getter/fetcher    retrieves data.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Common WizardSecondScreen.kt

```
L17: fun ImportConfigView(onProceed: (List<String>, String) -> Unit) {    Function    performs a feature-specific operation.
```

```
L82: fun GenerateConfigView(onProceed: (ExtractionConfigData) -> Unit) {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/ImportViewModel.kt

```

L23: data class ExtractionConfigData(    Data model holds structured fields.

L43: sealed class ImportStep {    Sealed hierarchy closed set of subclasses.

L44: data object NameAndDestination : ImportStep()    Singleton object shared utilities/DI.

L45: data object ChooseMethod : ImportStep()    Singleton object shared utilities/DI.

L46: data object MediaInput : ImportStep()    Singleton object shared utilities/DI.

L47: data class Configure(val mode: String) : ImportStep() // "IMPORT" or "GENERATE"    Data
model holds structured fields.

L48: data class Processing(val message: String = "AI is structuring your questions...") :
ImportStep()    Data model holds structured fields.

L49: data class Review(val questionCount: Int, val responseId: String) : ImportStep()    Data
model holds structured fields.

L50: data class Error(val message: String) : ImportStep()    Data model holds structured
fields.

L53: data object ExtractionIngest : ImportStep()    Singleton object shared utilities/DI.

L54: data class ExtractionOverview(val responseId: String, val response:
com.algorithmx.q_base.core_ai.brain.models.AiCollectionResponse) : ImportStep()    Data model
holds structured fields.

L57: data class Extracting(val source: String) : ImportStep()    Data model holds structured
fields.

L58: data class Editing(val extractedText: String) : ImportStep()    Data model holds
structured fields.

L59: data class Config(val extractedText: String, val targetId: String? = null) :
ImportStep()    Data model holds structured fields.

L60: data class Preview(val responseId: String, val response:
com.algorithmx.q_base.core_ai.brain.models.AiCollectionResponse) : ImportStep()    Data model
holds structured fields.

L61: data class Complete(val message: String) : ImportStep()    Data model holds structured
fields.

L65: class ImportViewModel @Inject constructor(    ViewModel manages UI state and

```

interactions.

L116: fun togglePaperType(type: String) { Function performs a feature-specific operation.

L142: fun setInitialSource(source: String?, targetId: String? = null) { Persists or updates data.

L153: private fun loadCollections() { Function performs a feature-specific operation.

L161: fun navigateTo(step: ImportStep) { Function performs a feature-specific operation.

L171: fun selectCategory(categoryId: String?) { Function performs a feature-specific operation.

L175: fun updateNewCollectionName(name: String) { Function performs a feature-specific operation.

L179: fun updateNewCollectionDescription(desc: String) { Function performs a feature-specific operation.

L183: fun updateCustomInstructions(instructions: String) { Function performs a feature-specific operation.

L187: fun selectMethod(method: String) { Function performs a feature-specific operation.

L191: fun onImagePicked(uri: Uri) { Function performs a feature-specific operation.

L199: fun onPdfPicked(uri: Uri) { Function performs a feature-specific operation.

L207: fun onRawTextUpdated(text: String) { Function performs a feature-specific operation.

L211: private fun handleExtractionResult(result: Result<String>) { Function performs a feature-specific operation.

L220: fun startDirectImport(types: List<String>, instructions: String) { Function performs a feature-specific operation.

L238: fun startAiGeneration(config: ExtractionConfigData) { Function performs a feature-specific operation.

L256: private fun handleGenerationResult(result: Result<String>) { Function performs a feature-specific operation.

L274: fun promoteResponse(responseId: String, customName: String? = null, onFinished: (String, String?) -> Unit) { Function performs a feature-specific operation.

L295: fun addPdf(uri: Uri) { Function performs a feature-specific operation.

L321: fun addOcr(uri: Uri) { Function performs a feature-specific operation.

L346: fun addClipboard(text: String) { Function performs a feature-specific operation.

```

L359: fun removeDoc(id: String) {    Function    performs a feature-specific operation.

L364: fun startPaperExtraction() {    Function    performs a feature-specific operation.

L401: fun handleRetry() {    Function    performs a feature-specific operation.

L409: fun reset() {    Function    performs a feature-specific operation.

L435: fun currentStepNumber(): Int = when (val s = _uiState.value) {    Function    performs a
feature-specific operation.

L453: fun totalSteps(): Int = 5    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Extraction WizardThirdScreen.kt

```

L21: fun ExtractionWizardThirdScreen(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/content_import/ManualBuilderScreen.kt

```

L29: fun ManualBuilderScreen(    Function    performs a feature-specific operation.

L131: fun ManualQuestionItem(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/content_import/ImportWizardScreen.kt

```

L20: fun ImportWizardScreen(    Function    performs a feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Extraction WizardFirstScreen.kt

```
L22: data class ExtractedDocumentCard(    Data model holds structured fields.

L31: fun ExtractionWizardFirstScreen(    Function performs a feature-specific operation.

L243: fun ExtractedDocItem(    Function performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/QuestionEditorViewModel.kt

```
L15: data class QuestionEditorState(    Data model holds structured fields.

L29: class QuestionEditorViewModel @Inject constructor(    ViewModel manages UI state and
interactions.

L41: fun init(questionId: String?, setId: String) {    Function performs a feature-specific
operation.

L70: fun updateStem(stem: String) {    Function performs a feature-specific operation.

L74: fun updateOption(letter: String, text: String) {    Function performs a feature-specific
operation.

L81: fun updateCorrectAnswer(answer: String) {    Function performs a feature-specific
operation.

L85: fun updateExplanation(explanation: String) {    Function performs a feature-specific
operation.

L89: fun updateReferences(refs: String) {    Function performs a feature-specific operation.

L93: fun getAiAssist() {    Getter/fetcher retrieves data.

L107: fun generateAiExplanation() {    Generates content (often AI).

L129: fun applyAiExplanation() {    Function performs a feature-specific operation.

L137: fun discardAiExplanation() {    Function performs a feature-specific operation.

L141: fun clearAiSuggestions() {    Function performs a feature-specific operation.
```

```
L145: fun saveQuestion() {    Persists or updates data.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Common WizardFourthScreen.kt

```
L26: fun ReviewView(    Function    performs a feature-specific operation.
```

```
L227: fun ErrorView(message: String, onRetry: () -> Unit) {    Function    performs a  
feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Extraction WizardFourthScreen.kt

```
L27: fun ExtractionWizardFourthScreen(    Function    performs a feature-specific operation.
```

```
L281: fun CountBadge(label: String, count: Int, color: Color, modifier: Modifier = Modifier)  
{    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/content_import/Common WizardFirstScreen.kt

```
L26: fun CommonWizardFirstScreen(    Function    performs a feature-specific operation.
```

```
L214: fun SmallSourceButton(icon: ImageVector, title: String, color: Color, modifier:  
Modifier = Modifier, onClick: () -> Unit) {    Function    performs a feature-specific  
operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/question/QuestionHeader.kt

```
L35: fun QuestionHeader(    Function    performs a feature-specific operation.  
  
L246: fun QuestionHeaderPreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/question/QuestionViewer.kt

```
L20: fun QuestionViewer(    Function    performs a feature-specific operation.  
  
L151: fun QuestionViewerPreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/question/TrueFalseToggle.kt

```
L15: fun TrueFalseToggle(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/question/OptionsList.kt

```
L29: fun OptionsList(    Function    performs a feature-specific operation.  
  
L341: fun OptionsListPreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/reusable/ReportDialog.kt

```
L13: fun ReportDialog(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/reusable/ProfileIconButton.kt

```
L24: fun ProfileIconButton(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/reusable/AiConfigSelector.kt

```
L16: fun AiConfigSelector(    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/ui/components/reusable/CommonComponents.kt

```
L32: fun UnifiedTopAppBar(    Function    performs a feature-specific operation.

L125: private fun AppAccessStateBadge(appAccessState: AppAccessState) {    Function    performs
a feature-specific operation.

L150: fun SessionCard(    Function    performs a feature-specific operation.

L196: fun SessionListItem(    Function    performs a feature-specific operation.

L230: fun SectionHeader(    Function    performs a feature-specific operation.

L272: fun SessionCardPreview() {    Function    performs a feature-specific operation.

L289: fun SessionListItemPreview() {    Function    performs a feature-specific operation.

L308: fun SectionHeaderPreview() {    Function    performs a feature-specific operation.
```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/BrainConfigProvider.kt

L5: interface BrainConfigProvider { Interface contract for implementations.

L6: suspend fun getApiKey(provider: BrainProvider): String Getter/fetcher retrieves data.

app/src/main/java/com/algorithmx/q_base/core_ai/brain/BrainDataStore Manager.kt

L24: class BrainDataStoreManager @Inject constructor(Class app component; see source for details.

L29: private object PreferencesKeys { Singleton object shared utilities/DI.

L73: suspend fun markSeedAsApplied() { Function performs a feature-specific operation.

L79: suspend fun resetSeedFlag() { Function performs a feature-specific operation.

L85: suspend fun saveEngineConfiguration(Persists or updates data.

L99: suspend fun incrementUsageStats(tokens: Int) { Function performs a feature-specific operation.

L108: suspend fun saveThemeMode(mode: String) { Persists or updates data.

L114: suspend fun saveNotificationsEnabled(enabled: Boolean) { Persists or updates data.

L120: suspend fun saveTaskConfig(task: BrainTask, config: TaskConfig) { Persists or updates data.

L135: suspend fun setMasterAiFreeze(freeze: Boolean) { Persists or updates data.

app/src/main/java/com/algorithmx/q_base/core_ai/brain/AiUsageLogger .kt

L6: interface AiUsageLogger { Interface contract for implementations.

L7: suspend fun logUsage(Function performs a feature-specific operation.

app/src/main/java/com/algorithmx/q_base/core_ai/brain/CommonAiService.kt

```

L13: class CommonAiService @Inject constructor(    AI component    orchestrates LLM/AI requests.

L22: suspend fun generateNoteStructure(    Generates content (often AI).

L58: suspend fun generateBlocksForTopic(    Generates content (often AI).

L73: private fun constructGenerationPrompt(    Function    performs a feature-specific
operation.

L104: private fun extractJsonFromResponse(response: String): String {    Function    performs a
feature-specific operation.

```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/AiBrainManager.kt

```

L24: class AiBrainManager @Inject constructor(    AI component    orchestrates LLM/AI requests.

L40: private suspend fun buildBrain(modelName: String): AiBrain? {    Function    performs a
feature-specific operation.

L67: suspend fun askBrain(task: BrainTask, prompt: String): Result<String> {    Function
performs a feature-specific operation.

L131: suspend fun streamFromBrain(task: BrainTask, prompt: String): Flow<String> {    Returns
observable/Flow for reactive updates.

```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/models/BrainTask.kt

```

L3: enum class BrainTask(val displayName: String) {    Enumeration    named constants.

```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/models/AiResponseModels.kt

```
L6: data class AiQuestion(    Data model    holds structured fields.

L15: data class AiOption(    Data model    holds structured fields.

L22: data class AiAnswer(    Data model    holds structured fields.

L29: data class AiCollectionResponse(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/models/AiModels.kt

```
L6: enum class BlockType {    Enumeration    named constants.

L16: data class AiGeneratedBlock(    Data model    holds structured fields.

L22: data class AiGeneratedTab(    Data model    holds structured fields.

L27: data class NoteContext(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/models/BrainModels.kt

```
L10: data class BrainConfig(    Data model    holds structured fields.

L17: data class TaskConfig(    Data model    holds structured fields.

L24: data class StoredBrainConfig(    Data model    holds structured fields.
```

app/src/main/java/com/algorithmx/q_base/core_ai/brain/di/AiCoreModule.kt

L16: object AiCoreModule { Singleton object shared utilities/DI.

L20: fun provideAiBrainManager(Function performs a feature-specific operation.

L30: fun provideCommonAiService(aiBrainManager: AiBrainManager): CommonAiService { Function performs a feature-specific operation.